



T.C.

BİTLİS EREN ÜNİVERSİTESİ
MÜHENDİSLİK-MİMARLIK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BMU1208 — WEB TABANLI PROGRAMLAMA

FİNAL PROJESİ

VANILLA JS KANBAN (TRELLO KLONU)

«Trello'nun sade, çevrimdışı çalışabilen, tamamen tarayıcıda yaşayan alternatifi»

Hazırlayan

ZEKERİYA KALKAN

Öğrenci No: 23080410034

Ders Yürütücüsü

Dr. Öğr. Üyesi Davut ARI

BİTLİS — 2025-2026 BAHAR

BEYAN

Bu raporda sunulan «Vanilla JS Kanban (Trello Klonu)» başlıklı çalışmanın tamamen tarafımdan hazırlanmış, bilimsel etik kurallarına uygun olarak yürütülmüş ve tüm alıntılar ile kaynaklar kaynakça bölümünde açıkça belirtilmiş olduğunu beyan ederim.

Geliştirme sürecinde yapay zekâ destekli araçlardan (Claude Code, GitHub Copilot, Cursor vb.) yardım alınmış olmakla birlikte; mimari kararlar, kullanılan teknoloji seçimleri, ürün yönetimi belgesi (PRD), kullanıcı araştırması ve iş modeli değerlendirmeleri tamamen kendi araştırmam ve takdirim sonucu oluşturulmuştur. AI araçlarından üretilen kod parçaları gözden geçirilmiş, gerekli iyileştirmeler yapılmış ve proje bütününe entegre edilmiştir.

ZEKERİYA KALKAN

Öğrenci No: 23080410034

ÖZET

VANILLA JS KANBAN (TRELLO KLONU)

Bu çalışma kapsamında, bireysel kullanıcıların ve küçük ekiplerin görevlerini görsel olarak planlayabilmesini sağlayan web tabanlı bir Kanban Board uygulaması geliştirilmiştir. Günümüzde görev yönetimi için kullanılan Trello, Notion ve Jira gibi araçlar güçlü özellikler sunsa da, çoğu zaman internet bağlantısına bağımlı çalışmakta, bazı gelişmiş özellikleri ücretli olarak sunmakta ve basit kullanım senaryoları için gereğinden fazla karmaşık olabilmektedir. Özellikle öğrenciler, freelancer çalışanlar ve küçük ekipler için sade, hızlı ve çevrimdışı kullanılabilen bir görev yönetim aracına ihtiyaç duyulmaktadır. Bu ihtiyaç doğrultusunda geliştirilen “Vanilla JS Kanban (Trello Klonu)” uygulaması, kullanıcıların görevlerini kolonlar halinde organize etmelerine, kartlar oluşturmalarına ve iş akışlarını kolayca takip etmelerine olanak sağlamaktadır.

Proje, tamamen istemci taraflı çalışacak şekilde HTML5, CSS3 ve Vanilla JavaScript (ES2024) teknolojileri kullanılarak geliştirilmiştir. Uygulama içerisinde Web Components tabanlı modüler yapı, merkezi state yönetimi, HTML5 Drag & Drop API, localStorage ile veri kalıcılığı, JSON içe/dışa aktarma ve jsPDF ile PDF raporlama gibi modern web teknolojileri kullanılmıştır. Kullanıcı deneyimini artırmak amacıyla dark/light tema desteği, klavye kısayolları, arama ve filtreleme özellikleri ile responsive tasarım yaklaşımı uygulanmıştır. Sistem herhangi bir backend altyapısına ihtiyaç duymadan doğrudan modern web tarayıcıları üzerinde çalışmaktadır.

Geliştirilen uygulama sonucunda kullanıcıların görevlerini hızlı, sezgisel ve çevrimdışı şekilde yönetebileceği işlevsel bir Kanban Board sistemi ortaya çıkarılmıştır. Yapılan Lighthouse ölçümlerinde performans ve erişilebilirlik açısından başarılı sonuçlar elde edilmiş, veri kalıcılığı ve kullanıcı deneyimi hedefleri karşılanmıştır. Proje, yalnızca bir ders çalışması olmanın ötesinde; modüler mimarisi, açık kaynak yaklaşımı ve genişletilebilir yapısı sayesinde gelecekte kullanıcı doğrulama, bulut senkronizasyonu ve gerçek zamanlı iş birliği gibi özelliklerle geliştirilebilecek bir ürün altyapısı sunmaktadır.

Anahtar Kelimeler: Kanban Board, görev yönetimi, üretkenlik, Vanilla JavaScript, HTML5, CSS3, Web Components, localStorage, Drag and Drop API, JSON Import/Export, jsPDF, çevrimdışı çalışma, PRD.

Bu çalışma kapsamında, bireysel kullanıcıların ve küçük ekiplerin görevlerini görsel olarak planlayabilmesini sağlayan web tabanlı bir Kanban Board uygulaması geliştirilmiştir. Günümüzde görev yönetimi için kullanılan Trello, Notion ve Jira gibi araçlar güçlü özellikler sunsa da, çoğu zaman internet bağlantısına bağımlı çalışmakta, bazı gelişmiş özellikleri ücretli olarak sunmakta ve basit kullanım senaryoları için gereğinden fazla karmaşık olabilmektedir. Özellikle öğrenciler, freelancer çalışanlar ve küçük ekipler için sade, hızlı ve çevrimdışı kullanılabilen bir görev yönetim aracına ihtiyaç duyulmaktadır. Bu ihtiyaç doğrultusunda geliştirilen “Vanilla JS Kanban (Trello Klonu)” uygulaması, kullanıcıların görevlerini kolonlar halinde organize etmelerine, kartlar oluşturmalarına ve iş akışlarını kolayca takip etmelerine olanak sağlamaktadır.

Proje, tamamen istemci taraflı çalışacak şekilde HTML5, CSS3 ve Vanilla JavaScript (ES2024) teknolojileri kullanılarak geliştirilmiştir. Uygulama içerisinde Web Components tabanlı modüler yapı, merkezi state yönetimi, HTML5 Drag & Drop API, localStorage ile veri kalıcılığı, JSON içe/dışa aktarma ve jsPDF ile PDF raporlama gibi modern web teknolojileri kullanılmıştır. Kullanıcı deneyimini artırmak amacıyla dark/light tema desteği, klavye kısayolları, arama ve filtreleme özellikleri ile responsive tasarım yaklaşımı uygulanmıştır. Sistem herhangi bir backend altyapısına ihtiyaç duymadan doğrudan modern web tarayıcıları üzerinde çalışmaktadır.

Geliştirilen uygulama sonucunda kullanıcıların görevlerini hızlı, sezgisel ve çevrimdışı şekilde yönetebileceği işlevsel bir Kanban Board sistemi ortaya çıkarılmıştır. Yapılan Lighthouse ölçümlerinde performans ve erişilebilirlik açısından başarılı sonuçlar elde edilmiş, veri kalıcılığı ve kullanıcı deneyimi hedefleri karşılanmıştır. Proje, yalnızca bir ders çalışması olmanın ötesinde; modüler mimarisi, açık kaynak yaklaşımı ve genişletilebilir yapısı sayesinde gelecekte kullanıcı doğrulama, bulut senkronizasyonu ve gerçek zamanlı iş birliği gibi özelliklerle geliştirilebilecek bir ürün altyapısı sunmaktadır.

Anahtar Kelimeler: Kanban Board, görev yönetimi, üretkenlik, Vanilla JavaScript, HTML5, CSS3, Web Components, localStorage, Drag and Drop API, JSON Import/Export, jsPDF, çevrimdışı çalışma, PRD.

ABSTRACT

VANILLA JS KANBAN (TRELLO KLONU)

This study presents a web-based Kanban Board application developed to help individual users and small teams visually organize and manage their tasks. Although widely used tools such as Trello, Notion, and Jira provide rich functionality, they often require internet connectivity, restrict advanced features to paid plans, and may be unnecessarily complex for simple task management scenarios. Students, freelancers, and small teams frequently need a lightweight, fast, and privacy-friendly solution that works directly in the browser. To address this need, the “Vanilla JS Kanban” application was developed as a Trello-like task management tool that enables users to create task cards, organize them into columns, and track their workflows efficiently.

The application was implemented entirely on the client side using HTML5, CSS3, and Vanilla JavaScript (ES2024). The system architecture includes a modular component-based structure, centralized state management, HTML5 Drag and Drop API, localStorage-based data persistence, JSON import/export, and PDF reporting with jsPDF. Additional features such as dark/light theme support, keyboard shortcuts, filtering, and responsive design were incorporated to improve usability. Since the application does not require any backend infrastructure, it can run directly on modern web browsers without installation.

As a result, a fully functional Kanban Board system was developed that allows users to manage tasks quickly, intuitively, and offline. Performance and accessibility evaluations using Lighthouse demonstrated successful outcomes, and the application met its core usability and persistence goals. Beyond being a course project, the resulting system provides a scalable foundation that can be extended in the future with features such as user authentication, cloud synchronization, and real-time collaboration.

Keywords: Kanban Board, task management, productivity, Vanilla JavaScript, HTML5, CSS3, Web Components, localStorage, Drag and Drop API, JSON Import/Export, jsPDF, offline-first, PRD.

İÇİNDEKİLER

[Bu sayfa Microsoft Word'de açıldığında «Evet, bu belgedeki tüm alanları güncelle» seçeneğine tıklayınız. Böylece içindekiler otomatik olarak güncellenecektir. Ctrl + A → F9 kısayolu da tüm alanları günceller.]

1. GİRİŞ	18
1.1. Problem Tanımı	18
1.2. Projenin Amacı ve Kapsamı	18
1.3. Raporun Yapısı	19
2. GEREKSİNİM ANALİZİ — PRD	21
2.1. Yönetici Özeti (Executive Summary)	21
2.2. Hedef Kitle ve Persona	21
2.3. Jobs To Be Done (JTBD)	23
2.3 Jobs To Be Done (JTBD)	23
2.4. Ana Özellikler ve Kullanıcı Hikâyeleri (User Stories)	24
User Story 1 — Kanban Board Oluşturma	25
User Story 2 — Kart Oluşturma	25
User Story 3 — Kart Düzenleme	25
User Story 4 — Checklist Kullanımı	25
User Story 5 — Sürükle-Bırak	26
User Story 6 — Arama ve Filtreleme	26
User Story 7 — Tema Değiştirme	26
User Story 8 — Keyboard Shortcuts	27
User Story 9 — JSON Import / Export	27
User Story 10 — PDF Raporlama	27
2.5. İşlevsel Olmayan Gereksinimler (NFR)	28
2.6 Kapsam Dışı (Out-of-Scope)	29
3. PİYASA VE REKABET ANALİZİ	30
3.1. Pazar Büyüklüğü (TAM / SAM / SOM)	30

TAM (Total Addressable Market)	30
SAM (Serviceable Available Market)	30
SOM (Serviceable Obtainable Market)	30
Genel Değerlendirme	31
3.2. Rakip Karşılaştırma Matrisi.....	31
3.2 Rakip Karşılaştırma Matrisi.....	31
Genel Değerlendirme	32
3.3. Detaylı Rakip İncelemesi.....	33
1. Trello.....	33
2. Notion	33
3. Jira.....	34
Vanilla JS Kanban'ın Konumu	34
Sonuç	35
3.4. SWOT Analizi	35
Genel Değerlendirme	36
3.5. Positioning Statement (Konumlandırma)	36
Türkçe Açıklama	37
4. TEKNOLOJİ YIĞINI (TECH STACK).....	38
4.1. Katmanlar — Özet Tablosu	38
4.1 Katmanlar — Özet Tablosu	38
4.2. Frontend.....	39
4.2.1. Vanilla JavaScript (ES2024).....	39
Neden Seçildi?	39
Temel Özellikler	39
Alternatifler ve Neden Seçilmedi?.....	39
Trade-Off'lar.....	40
Öğrenme Kaynakları.....	40

Projedeki Rolü	40
4.2.2. Web Components (HTM5)	40
Neden Seçildi?	40
Temel Özellikler	41
Alternatifler ve Neden Seçilmedi?.....	41
Trade-Off'lar.....	41
Öğrenme Kaynakları.....	41
Projedeki Rolü	41
4.2.3. Tailwind CSS 3.....	42
Neden Seçildi?	42
Temel Özellikler	42
Alternatifler ve Neden Seçilmedi?.....	42
Trade-Off'lar.....	42
Öğrenme Kaynakları.....	43
Projedeki Rolü	43
4.3. State & Drag-Drop.....	43
4.3.1. State Yönetimi (Merkezi App.data ve App.users Yapısı)	43
Neden Seçildi?	43
Temel Özellikler	43
Alternatifler ve Neden Seçilmedi?.....	44
Trade-Off'lar.....	44
Öğrenme Kaynakları.....	44
Projedeki Rolü	44
4.3.2. HTM5 Drag & Drop API.....	45
Neden Seçildi?	45
Temel Özellikler	45
Alternatifler ve Neden Seçilmedi?.....	45

Trade-Off'lar.....	45
Öğrenme Kaynakları.....	46
Projedeki Rolü	46
4.4. Depolama	46
4.4.1. JSON Import / Export.....	46
Neden Seçildi?	46
Temel Özellikler	46
Alternatifler ve Neden Seçilmedi?.....	47
Trade-Off'lar.....	47
Öğrenme Kaynakları.....	47
Projedeki Rolü	47
4.4.2. LocalStorage (preferences)	48
Neden Seçildi?	48
Temel Özellikler	48
Alternatifler ve Neden Seçilmedi?.....	48
Trade-Off'lar.....	49
Öğrenme Kaynakları.....	49
Projedeki Rolü	49
4.5. PWA	49
4.5.1. jsPDF (Workbox).....	49
Neden Seçildi?	49
Temel Özellikler	50
Alternatifler ve Neden Seçilmedi?.....	50
Trade-Off'lar.....	50
Öğrenme Kaynakları.....	50
Projedeki Rolü	51
4.5.2. Web App Manifest (File API)	51

Neden Seçildi?	51
Temel Özellikler	51
Alternatifler ve Neden Seçilmedi?.....	51
Trade-Off'lar.....	52
Öğrenme Kaynakları.....	52
Projedeki Rolü	52
4.6. Hosting.....	52
4.6.1. Modern Web Tarayıcıları	52
Neden Seçildi?	52
Temel Özellikler	53
Alternatifler ve Neden Seçilmedi?.....	53
Trade-Off'lar.....	53
Öğrenme Kaynakları.....	53
Projedeki Rolü	54
4.7. Reddedilen Teknoloji Kararları	54
Genel Değerlendirme	55
5. SİSTEM MİMARİSİ	56
5.1. Yüksek Seviye Mimari (C4 — Level 1: Context)	57
5.2. Container Seviyesi (C4 — Level 2).....	58
5.3. Sıralı (Sequence) Diyagramları	59
5.4. Deployment Topolojisi	60
5.5. Mimari Karar Kayıtları (Architecture Decision Records — ADR).....	60
6. VERİ MODELİ VE API TASARIMI	62
6.1. ER Diyagramı	62
6.2. Tablo Tanımları	62
6.2 Tablo Tanımları	62
6.3. İndeks Stratejisi	64

6.4. REST API Endpoint Listesi	65
6.5 Authentication ve Authorization.....	66
Authentication (Kimlik Doğrulama).....	66
Authorization (Yetkilendirme)	67
Güvenlik Önlemleri	67
6.6. Rate Limiting Politikası	67
7. KULLANICI ARAYÜZÜ TASARIMI.....	69
7.1. Bilgi Mimarisi (Sitemap).....	69
7.2. Kullanıcı Akışı (User Flow)	69
7.3. Design System	70
7.3 Design System	70
7.4. Wireframe'ler ve Mockuplar	71
7.5. Responsive Tasarım Stratejisi.....	72
Responsive Tasarım Kararları	72
7.6. Erişilebilirlik (a11y).....	73
7.6 Erişilebilirlik (a11y).....	73
Uygulanan Erişilebilirlik Önlemleri	73
Test ve Doğrulama.....	74
Genel Değerlendirme	74
8. GÜVENLİK, PERFORMANS VE TEST	75
8.1. Güvenlik (OWASP Top 10)	75
8.2. Performans Hedefleri.....	75
8.3. Test Stratejisi	76
9. MALİYET, GELİR MODELİ VE GTM.....	77
9.1. Business Model Canvas	77
9.2. Gelir Modeli.....	77
9.2 Gelir Modeli.....	77

9.3. Maliyet Analizi	78
9.4. Unit Economics	79
9.5. Go-to-Market (GTM) Stratejisi	79
Hedef Kitle.....	79
İlk 100 Kullanıcıya Ulaşma Planı.....	79
İçerik Pazarlama Stratejisi	80
Viral Büyüme Mekanizmaları	80
Başarı Metrikleri.....	80
Genel Değerlendirme.....	80
10. UYGULAMA VE GELİŞTİRME	81
10.1. Kurulum ve Çalıştırma	81
Sistem Gereksinimleri	81
Kaynak Kodun İndirilmesi	81
Uygulamanın Çalıştırılması	81
İlk Çalıştırma	82
Klasör Yapısı	82
Genel Değerlendirme.....	82
10.2. Ekran Görüntüleri	82
10.3. Kullanılan AI Araçları ve Katkı Oranı	86
11. SONUÇ VE DEĞERLENDİRME	88
11.1. Proje Hakkında Genel Değerlendirme.....	88
11.2. Karşılaşılan Zorluklar ve Çözümleri	89
11.3. Gelecek Çalışmalar (Future Work).....	89
11.4. Kazanılan Yetkinlikler.....	89
Teknik Kazanımlar	90
Süreçsel Kazanımlar	90
Genel Değerlendirme.....	90

KAYNAKÇA	92
----------------	----

ŞEKİLLER LİSTESİ

Şekil 1: Sistem Context Diyagramı (C4 — Level 1).....	57
Şekil 2: Container Diyagramı (C4 — Level 2).....	58
Şekil 3: Login Akışı Sıralı Diyagramı	59
Şekil 4: Production Deployment Topolojisi	60
Şekil 5: Veritabanı ER Diyagramı	62
Şekil 6: Uygulama Site Haritası	69
Şekil 7: Ana Kullanıcı Akışı (User Flow)	69
Şekil 8: Landing Sayfası Wireframe.....	71
Şekil 9: Dashboard Mockup	71
Şekil 10: Test Piramidi	76
Şekil 11: Landing / Ana Sayfa.....	82
Şekil 12: Kayıt / Giriş Ekranı	83
Şekil 13: Dashboard — Boş State	83
Şekil 14: Light Mode Görünüm.....	84
Şekil 15: Ana Filtreleme Mekanizması	84
Şekil 16: Kısayol Gösterimleri	85
Şekil 17: Mobile Görünüm	85
Şekil 18: Detaylı Kart Gösterimi	86

KISALTMALAR VE SİMGELER

Kısaltma	Açıklama
API	Application Programming Interface — Uygulama Programlama Arayüzü
ADR	Architecture Decision Record — Mimari Karar Kaydı
BaaS	Backend-as-a-Service
BMC	Business Model Canvas — İş Modeli Kanvası
CI/CD	Continuous Integration / Continuous Deployment
CRUD	Create, Read, Update, Delete
DB	Database — Veritabanı
E2E	End-to-End (Test)
ERD	Entity-Relationship Diagram — Varlık-İlişki Diyagramı
GTM	Go-to-Market — Pazara Giriş
JTBD	Jobs To Be Done — Yapılacak İşler çerçevesi
JWT	JSON Web Token
KVKK	Kişisel Verilerin Korunması Kanunu
LTV	Customer Lifetime Value
MAU	Monthly Active Users
NFR	Non-Functional Requirements — İşlevsel Olmayan Gereksinimler
OWASP	Open Worldwide Application Security Project
PRD	Product Requirements Document
PWA	Progressive Web Application
REST	Representational State Transfer
RBAC	Role-Based Access Control
RLS	Row Level Security

SaaS	Software-as-a-Service
SSE	Server-Sent Events
SSL/TLS	Secure Sockets Layer / Transport Layer Security
SWOT	Strengths, Weaknesses, Opportunities, Threats
TDD	Technical Design Document / Test-Driven Development
TCO	Total Cost of Ownership
UX/UI	User Experience / User Interface
WCAG	Web Content Accessibility Guidelines

1. GİRİŞ

1.1. Problem Tanımı

Günümüzde bireysel kullanıcılar ve küçük ekipler, görev takibi ve proje yönetimi süreçlerini dijital platformlar üzerinden yürütmektedir. Özellikle yazılım geliştirme, içerik üretimi, eğitim ve ekip koordinasyonu gibi alanlarda, görevlerin düzenli bir şekilde takip edilmesi büyük önem taşımaktadır. Ancak görev sayısının artmasıyla birlikte süreçlerin yönetimi zorlaşmakta, kullanıcılar yapılacak işleri takip etmekte ve iş akışını organize etmekte çeşitli problemler yaşamaktadır. Bu durum, zaman kaybına, verimlilik düşüşüne ve iş süreçlerinde düzensizliğe neden olmaktadır.

Bu problemlerin çözümünde Trello, Notion ve Jira gibi popüler proje yönetim araçları yaygın olarak kullanılmaktadır. Ancak bu sistemler özellikle bireysel kullanıcılar ve küçük ekipler açısından bazı dezavantajlara sahiptir. Mevcut platformların büyük bölümü sürekli internet bağlantısına ihtiyaç duymakta, bazı özellikleri ücretli paketler altında sunulmakta ve karmaşık arayüz yapıları nedeniyle kullanıcı deneyimini zorlaştırabilmektedir. Ayrıca birçok kullanıcı, yalnızca temel görev takibi yapmak istemesine rağmen kapsamlı ve ağır sistemlerle karşılaşmaktadır. Bu durum, özellikle hızlı çalışan, sade ve çevrimdışı kullanılabilen bir görev yönetim sistemine olan ihtiyacı artırmaktadır.

Bu proje kapsamında geliştirilen “Vanilla JS Kanban (Trello Klonu)” uygulaması, söz konusu problemlere çözüm sunmak amacıyla tasarlanmıştır. Geliştirilen sistem; kullanıcıların görevlerini kolon bazlı olarak organize edebilmesini, görev kartları oluşturabilmesini, sürükle-bırak yöntemiyle iş akışını yönetebilmesini ve verilerini tarayıcı üzerinde saklayabilmesini sağlamaktadır. Uygulama tamamen istemci tarafı çalışacak şekilde geliştirilmiş olup herhangi bir backend altyapısına ihtiyaç duymadan kullanılabilir. Ayrıca çevrimdışı çalışma desteği, klavye kısayolları, tema yönetimi, veri dışı aktarma ve kullanıcı dostu arayüz gibi özelliklerle desteklenerek modern ve erişilebilir bir görev yönetim deneyimi sunulması hedeflenmiştir.

1.2. Projenin Amacı ve Kapsamı

Bu projenin temel amacı, kullanıcıların görevlerini görsel ve sistematik bir şekilde yönetebileceği, hızlı, sade ve kullanıcı dostu bir Kanban Board uygulaması geliştirmektir. Geliştirilen sistem ile birlikte bireysel kullanıcıların ve küçük ekiplerin görevlerini daha verimli organize edebilmesi, süreçlerini takip edebilmesi ve iş akışlarını daha düzenli hale getirebilmesi hedeflenmektedir. Proje kapsamında, modern web teknolojileri kullanılarak tamamen istemci tarafı çalışan ve herhangi bir sunucu altyapısına ihtiyaç duymayan bir görev yönetim sistemi tasarlanmıştır.

Uygulama içerisinde kullanıcılar; kolon oluşturabilmekte, görev kartları ekleyebilmekte, kart detaylarını düzenleyebilmekte ve görevleri sürükle-bırak yöntemiyle farklı aşamalar arasında taşıyabilmektedir. Bununla birlikte kartlar üzerinde açıklama, etiket, kullanıcı

atama, son tarih belirleme ve checklist yönetimi gibi gelişmiş özellikler sunulmaktadır. Kullanıcı deneyimini artırmak amacıyla dark/light tema desteği, klavye kısayolları, arama ve filtreleme sistemi gibi yardımcı özellikler de projeye dahil edilmiştir. Ayrıca sistem, localStorage tabanlı veri saklama mekanizması sayesinde çevrimdışı çalışabilmekte ve kullanıcı verilerini tarayıcı üzerinde kalıcı olarak saklayabilmektedir.

Projenin MVP (Minimum Viable Product) kapsamında aşağıdaki temel özellikler yer almaktadır:

- Kanban board yapısı (To Do / In Progress / Done)
- Kart oluşturma, düzenleme ve silme işlemleri
- Sürükle-bırak (drag-and-drop) desteği
- Kart detay yönetimi
- Etiketleme sistemi
- Kullanıcı atama sistemi
- Son tarih yönetimi
- Checklist desteği
- Arama ve filtreleme özellikleri
- Dark / Light tema desteği
- Keyboard shortcuts desteği
- JSON veri içe/dışa aktarma
- PDF rapor oluşturma
- localStorage ile veri kalıcılığı

Proje kapsamında bilinçli olarak kapsam dışı bırakılan bazı özellikler de bulunmaktadır. Bu yaklaşım, proje kapsamının kontrol altında tutulmasını ve temel işlevlerin daha stabil şekilde geliştirilmesini amaçlamaktadır. MVP sürümünde kapsam dışı bırakılan özellikler aşağıdaki gibidir:

- Sunucu tabanlı backend sistemi
- Gerçek zamanlı çoklu kullanıcı senkronizasyonu
- Kullanıcı giriş sistemi (authentication)
- Bulut tabanlı veri senkronizasyonu
- Gelişmiş mobil sürükle-bırak optimizasyonu
- Yetkilendirme ve rol yönetimi sistemi
- Gerçek zamanlı bildirim altyapısı

Sonuç olarak bu proje, modern kullanıcı ihtiyaçlarına cevap verebilen, çevrimdışı çalışabilen, genişletilebilir ve kullanıcı deneyimi odaklı bir görev yönetim sistemi geliştirmeyi amaçlamaktadır.

1.3. Raporun Yapısı

Bu rapor, geliştirilen “Vanilla JS Kanban (Trello Klonu)” projesinin analiz, tasarım, geliştirme ve değerlendirme süreçlerini kapsamlı şekilde açıklamak amacıyla hazırlanmıştır. Raporun ikinci bölümünde proje için hazırlanan gereksinim analizi ve Ürün Gereksinimleri Belgesi (PRD) kapsamında hedef kullanıcı kitlesi, kullanıcı ihtiyaçları, kullanıcı hikâyeleri ve işlevsel gereksinimler ele alınmıştır.

Üçüncü bölümde proje için gerçekleştirilen pazar ve rakip analizleri sunulmuş, mevcut proje yönetim araçları incelenerek sistemin güçlü ve zayıf yönleri değerlendirilmiştir. Dördüncü bölümde projede kullanılan teknoloji yığını detaylı olarak açıklanmış, tercih edilen teknolojilerin kullanım amaçları ve sistem içerisindeki rolleri incelenmiştir.

Beşinci bölümde sistem mimarisi açıklanarak uygulamanın genel yapısı, veri akışı ve modüler mimari yaklaşımı ele alınmıştır. Altıncı bölümde veri modeli, veri saklama yapısı ve API tasarımı incelenmiştir. Yedinci bölümde kullanıcı arayüzü tasarımı, kullanıcı deneyimi yaklaşımı, responsive tasarım stratejileri ve erişilebilirlik çalışmaları açıklanmıştır.

Sekizinci bölümde güvenlik, performans ve test süreçleri değerlendirilmiş, uygulamanın doğrulama ve hata yönetimi mekanizmaları ele alınmıştır. Dokuzuncu bölümde maliyet analizi, gelir modeli ve ürünün pazara giriş stratejisi değerlendirilmiştir. Onuncu bölümde uygulamanın geliştirme süreci, kurulum aşamaları ve ekran görüntüleri sunulmuştur.

Son bölümde ise proje genel olarak değerlendirilmiş, geliştirme sürecinde karşılaşılan problemler, elde edilen kazanımlar ve gelecekte gerçekleştirilebilecek geliştirmeler açıklanmıştır.

2. GEREKSİNİM ANALİZİ — PRD

Ürün Gereksinimleri Belgesi (Product Requirements Document), projenin neden, kim için, ne yapacağı sorularını cevaplayan ve geliştirme ekibi ile ürün/iş tarafı arasında «sözleşme» işlevi gören dokümandır.

2.1. Yönetici Özeti (Executive Summary)

Bu proje kapsamında, kullanıcıların görevlerini görsel ve düzenli bir şekilde yönetebilmesini sağlayan web tabanlı bir Kanban Board uygulaması geliştirilmiştir. “Vanilla JS Kanban (Trello Klonu)” adı verilen bu sistem, tamamen istemci taraflı çalışan, sade yapıya sahip ve çevrimdışı kullanılabilen bir görev yönetim platformu olarak tasarlanmıştır. Uygulama içerisinde kullanıcılar; kolon oluşturabilmekte, görev kartları ekleyebilmekte, sürükle-bırak yöntemiyle görevlerini organize edebilmekte ve görev detaylarını yönetebilmektedir. Ayrıca sistem; dark/light tema desteği, klavye kısayolları, JSON veri aktarımı ve PDF raporlama gibi kullanıcı deneyimini artıran ek özelliklerle desteklenmiştir.

Günümüzde görev yönetimi ve proje takip süreçleri dijital araçlar üzerinden yürütülmesine rağmen, mevcut sistemlerin büyük bölümü internet bağlantısına bağımlı çalışmakta ve birçok kullanıcı için gereğinden fazla karmaşık yapılar sunmaktadır. Özellikle bireysel kullanıcılar ve küçük ekipler, daha sade, hızlı ve düşük kaynak tüketimine sahip çözümlere ihtiyaç duymaktadır. Bu doğrultuda geliştirilen proje, herhangi bir framework bağımlılığı olmadan yalnızca HTML5, CSS3 ve Vanilla JavaScript teknolojileri kullanılarak geliştirilmiş ve temel web teknolojilerinin etkin kullanımını göstermeyi amaçlamıştır. Ayrıca localStorage tabanlı veri kalıcılığı sayesinde uygulamanın çevrimdışı çalışabilmesi sağlanmıştır.

Projenin başarısı; kullanıcı deneyimi, sistem performansı, veri kalıcılığı ve kullanılabilirlik kriterleri üzerinden değerlendirilmiştir. Geliştirilen sistemin temel hedefi, kullanıcıların görev yönetim süreçlerini daha hızlı ve verimli hale getirmek, sade bir arayüz üzerinden sezgisel bir kullanım deneyimi sunmak ve modern web uygulama geliştirme prensiplerini uygulamalı olarak ortaya koymaktır. Proje sonucunda, genişletilebilir mimariye sahip, modüler yapıda ve kullanıcı odaklı çalışan bir Kanban Board sistemi başarıyla geliştirilmiştir.

2.2. Hedef Kitle ve Persona

Birincil segment: Bireysel kullanıcı veya 2-5 kişilik mikro ekip

Tablo 1: Persona 1 Kartı

Alan	Değer
------	-------

Ad	Dijital Emir
Yaş / Şehir	22 / Bitlis
Rol / Meslek	Bilgisayar Mühendisliği Öğrencisi
Teknoloji kullanımı	Windows dizüstü bilgisayar, Chrome tarayıcı, VS Code, Discord, GitHub
Günlük rutin (1 cümle)	Gün içerisinde ders, proje geliştirme ve bireysel çalışma süreçlerini birlikte yürütmektedir.
Ana hedef	Ders projelerini, kişisel görevlerini ve günlük planlarını düzenli şekilde takip etmek
Pain Points (3)	1. Görevleri takip ederken dağınıklık yaşaması 2. Karmaşık proje yönetim araçlarının gereksiz gelmesi 3. İnternet bağlantısı olmadığında mevcut sistemleri kullanamaması
Ürünümüzü ne zaman açar?	Ders çalışırken, proje geliştirirken veya günlük görev planlaması yaparken
Motto	“Basit ama hızlı çalışan bir sistem istiyorum.”

Tablo 2: Persona 2 Kartı

Alan	Değer
Ad	Freelancer Zeynep
Yaş / Şehir	28 / İstanbul
Rol / Meslek	Freelancer Grafik Tasarımcı
Teknoloji kullanımı	MacBook, Figma, Notion, Google Drive, Slack
Günlük rutin (1 cümle)	Aynı anda birden fazla müşteri işi ve teslim tarihini yönetmektedir.
Ana hedef	İş süreçlerini görsel şekilde takip ederek teslim tarihlerini kaçırmamak
Pain Points (3)	1. Birden fazla projeyi aynı anda yönetmekte zorlanması 2. Bazı proje yönetim araçlarının pahalı olması 3. Hızlı işlem yapmak isterken arayüz karmaşası yaşaması
Ürünümüzü ne zaman açar?	Günlük iş planlaması yaparken ve görev durumlarını kontrol ederken

Motto	“Görevlerimi tek ekrandan hızlıca yönetebilmeliyim.”
--------------	--

Tablo 3: Persona Özellik Karşılaştırma Tablosu

Özellik	Dijital Emir	Freelance Zeynep
Basit Arayüz	Yüksek	Yüksek
Sürükle-Bırak	Yüksek	Yüksek
Çevrimdışı Çalışma	Yüksek	Orta
Çoklu Board Desteği	Orta	Yüksek
Arama ve Filtreleme	Orta	Yüksek
Keyboard Shortcuts	Yüksek	Orta
PDF Raporlama	Düşük	Yüksek
JSON içe/Dışa Aktarma	Orta	Orta
Dak/Light Mode	Yüksek	Orta
Son Tarih Takibi	Orta	Yüksek

2.3. Jobs To Be Done (JTBD)

2.3 Jobs To Be Done (JTBD)

Jobs To Be Done (JTBD) yaklaşımı, kullanıcıların bir ürünü yalnızca özellikleri için değil, belirli bir işi gerçekleştirmek amacıyla “işe aldığını” ifade etmektedir. Bu yaklaşım, kullanıcıların gerçek ihtiyaçlarını ve üründen bekledikleri temel faydayı daha net bir şekilde ortaya koymaktadır. Geliştirilen Kanban Board uygulaması için belirlenen temel JTBD ifadeleri aşağıda sunulmuştur.

1. **When I’m managing multiple assignments and personal tasks, I want to organize them visually on a Kanban board, so I can track my progress and avoid missing important deadlines.**
2. **When I’m working without a stable internet connection, I want to access and update my tasks offline, so I can continue managing my work without interruption.**
3. **When I’m handling tasks with many subtasks, I want to break them down into checklist items, so I can monitor detailed progress more effectively.**
4. **When I’m preparing for upcoming deadlines, I want to assign due dates and labels to tasks, so I can prioritize my work and focus on the most critical items first.**

5. **When I need to archive or share my work, I want to export my board data as JSON or PDF, so I can back up my information and use it in different environments.**

Bu JTBD ifadeleri incelendiğinde, kullanıcıların temel beklentilerinin görevleri görsel olarak organize etmek, çevrimdışı çalışabilmek, detaylı görev takibi yapmak, önceliklendirme gerçekleştirmek ve verilerini güvenli bir şekilde dışa aktarabilmek olduğu görülmektedir. Proje kapsamında geliştirilen özellikler, bu ihtiyaçları doğrudan karşılayacak şekilde tasarlanmıştır.

2.4. Ana Özellikler ve Kullanıcı Hikâyeleri (User Stories)

Bu bölümde, proje kapsamında geliştirilen temel özellikler ve bu özelliklere ilişkin kullanıcı hikâyeleri sunulmaktadır. Kullanıcı hikâyeleri, ürünün hedef kullanıcılar açısından hangi problemi çözdüğünü ve hangi faydayı sağladığını ortaya koymaktadır. Ayrıca her kullanıcı hikâyesi için kabul kriterleri (acceptance criteria) tanımlanarak ilgili özelliğin hangi koşullarda tamamlanmış kabul edileceği belirtilmiştir.

Tablo 4: MVP Kapsamındaki Temel Özellikler

Özellik	Açıklama / Öncelik
Kanban Board (To Do / In Progress / Done)	Must-have / V1
Kart CRUD İşlemleri	Must-have / V1
Sürükle-Bırak Desteği	Must-have / V1
Kart Detay Yönetimi	Must-have / V1
Checklist Sistemi	Must-have / V1
Etiketleme Sistemi	Must-have / V1
Son Tarih Takibi	Must-have / V1
Arama ve Filtreleme	Must-have / V1
Dark / Light Tema	Must-have / V1
Keyboard Shortcuts	Should-have / V1
JSON Import / Export	Should-have / V1
PDF Raporlama	Could-have / V1

User Story 1 — Kanban Board Oluşturma

User Story:

As a user, I want to organize my tasks into columns such as To Do, In Progress and Done, so that I can track the status of my work visually.

Acceptance Criteria:

- Given the application is opened, when the board is loaded, then default columns are displayed.
- Given a task exists, when it is moved to another column, then its status is updated.
- Given the board is refreshed, when data is reloaded, then column structure is preserved.

User Story 2 — Kart Oluşturma

User Story:

As a user, I want to create a task card with a title, so that I can record a new task quickly.

Acceptance Criteria:

- Given a column is selected, when the user enters a valid title, then a new card is created.
- Given the card is created, when the page is refreshed, then the card remains visible.

User Story 3 — Kart Düzenleme

User Story:

As a user, I want to edit card details such as description, label and due date, so that I can manage tasks in more detail.

Acceptance Criteria:

- Given a card exists, when the detail modal is opened, then all editable fields are displayed.
- Given a field is changed, when the user saves, then the updates are persisted.

User Story 4 — Checklist Kullanımı

User Story:

As a user, I want to divide a task into subtasks, so that I can monitor detailed progress.

Acceptance Criteria:

- Given a card is open, when the user adds a checklist item, then it appears in the list.
- Given an item is marked complete, when progress is recalculated, then completion ratio is updated.

User Story 5 — Sürükle-Bırak

User Story:

As a user, I want to drag and drop cards between columns, so that I can update task status quickly.

Acceptance Criteria:

- Given a card is selected, when it is dropped into another column, then its new position is saved.
- Given the page is refreshed, when data is loaded, then the updated position is retained.

User Story 6 — Arama ve Filtreleme

User Story:

As a user, I want to search and filter cards, so that I can find specific tasks quickly.

Acceptance Criteria:

- Given multiple cards exist, when a keyword is entered, then only matching cards are shown.
- Given a label filter is selected, then only cards with that label are displayed.

User Story 7 — Tema Değiştirme

User Story:

As a user, I want to switch between dark and light themes, so that I can use the application comfortably in different environments.

Acceptance Criteria:

- Given the theme toggle is clicked, when the selection changes, then the interface theme updates.
- Given the page is refreshed, then the selected theme remains active.

User Story 8 — Keyboard Shortcuts

User Story:

As a power user, I want to perform common actions using keyboard shortcuts, so that I can work faster.

Acceptance Criteria:

- Given the application is active, when shortcut keys are pressed, then corresponding actions are executed.
- Given the help shortcut is pressed, then shortcut documentation is displayed.

User Story 9 — JSON Import / Export

User Story:

As a user, I want to export and import board data in JSON format, so that I can back up and restore my tasks.

Acceptance Criteria:

- Given board data exists, when export is triggered, then a JSON file is downloaded.
- Given a valid JSON file is imported, then board data is restored successfully.

User Story 10 — PDF Raporlama

User Story:

As a user, I want to generate a PDF report, so that I can archive and share the current board state.

Acceptance Criteria:

- Given board data exists, when PDF export is selected, then a report file is created.
- Given the file is opened, then board information is displayed in readable format.

Bu kullanıcı hikâyeleri, ürünün temel işlevlerini kullanıcı bakış açısından tanımlamakta ve her özelliğin doğrulanabilir kabul kriterleri ile desteklenmesini sağlamaktadır. Böylece geliştirme süreci daha sistematik hale getirilmiş ve her özelliğin kapsamı açık şekilde belirlenmiştir.

2.5. İşlevsel Olmayan Gereksinimler (NFR)

Tablo 5: NFR Matrisi

Kategori	Gereksinim	Nasıl ölçülecek?
Performans	Uygulamanın ilk yüklenme süresi 2.5 saniyenin altında olmalıdır.	Google Lighthouse
Performans	Kart oluşturma, güncelleme ve taşıma işlemleri kullanıcı açısından anlık gerçekleşmelidir.	Manuel kullanıcı testi
Kullanılabilirlik	Temel işlemler en fazla 3 tıklama ile tamamlanabilmelidir.	Kullanılabilirlik testi
Güvenilirlik	Sayfa yenileme sonrasında tüm veriler eksiksiz olarak korunmalıdır.	localStorage doğrulama testi
Uyumluluk	Chrome, Firefox, Edge ve Safari tarayıcılarında çalışmalıdır.	Tarayıcılar arası test
Erişilebilirlik	Uygulama klavye ile kullanılabilir olmalıdır.	Manuel keyboard testi
Erişilebilirlik	Renk kontrast oranları okunabilirliği desteklemelidir.	Lighthouse Accessibility
Güvenlik	Kullanıcı girdileri HTML olarak yorumlanmamalıdır.	XSS testi
Veri Taşınabilirliği	Veriler JSON formatında dışa ve içe aktarılabilir.	Import/Export testi
Kullanıcı Deneyimi	Tema tercihi ve kullanıcı ayarları kalıcı olarak saklanmalıdır.	localStorage testi
Çevrimdışı Kullanım	İnternet bağlantısı olmadan mevcut verilerle çalışmaya devam edebilmelidir.	Offline tarayıcı testi

Bu gereksinimler, uygulamanın yalnızca fonksiyonel açıdan değil, kalite özellikleri bakımından da kullanıcı beklentilerini karşılama hedeflemektedir. Özellikle performans, veri kalıcılığı ve çevrimdışı çalışma desteği, projenin temel değer önerisinin önemli bileşenleri arasında yer almaktadır. Bu nedenle geliştirme sürecinde tüm özellikler, belirtilen kalite kriterleri dikkate alınarak tasarlanmış ve test edilmiştir.

2.6 Kapsam Dışı (Out-of-Scope)

Proje geliştirme sürecinde kapsamın kontrollü bir şekilde yönetilebilmesi amacıyla, yalnızca temel kullanıcı ihtiyaçlarını karşılayan özellikler MVP (Minimum Viable Product) sürümüne dahil edilmiştir. Daha fazla geliştirme süresi, sunucu altyapısı veya ek entegrasyon gerektiren bazı özellikler ise bilinçli olarak kapsam dışında bırakılmıştır. Bu yaklaşım sayesinde, temel işlevlerin daha kararlı ve kullanıcı dostu şekilde tamamlanması hedeflenmiştir.

V1 sürümünde kapsam dışı bırakılan başlıca özellikler aşağıda listelenmiştir:

- Kullanıcı kayıt ve giriş sistemi (Authentication)
- Rol tabanlı yetkilendirme (RBAC)
- Sunucu tabanlı veritabanı entegrasyonu
- Bulut tabanlı veri senkronizasyonu
- Gerçek zamanlı çoklu kullanıcı desteği
- WebSocket tabanlı canlı güncellemeler
- E-posta ve push bildirim sistemi
- Gantt chart ve takvim görünümü
- Gelişmiş raporlama ve analitik modülü
- Mobil uygulama (iOS / Android)
- Gelişmiş dokunmatik sürükle-bırak optimizasyonları
- Üçüncü taraf entegrasyonları (Google Calendar, Slack, GitHub vb.)

Bu özelliklerin kapsam dışında bırakılmasının temel nedeni, projenin ders süresi içerisinde tamamlanabilir olmasını sağlamak ve temel Kanban işlevlerine odaklanmaktır. Bununla birlikte, geliştirilen modüler mimari sayesinde bu özelliklerin gelecekte sisteme eklenmesi mümkündür.

Özellikle kullanıcı doğrulama, bulut senkronizasyonu ve gerçek zamanlı iş birliği özellikleri, uygulamanın SaaS tabanlı bir ürüne dönüştürülmesi durumunda öncelikli geliştirme alanları olacaktır. Bu nedenle kapsam dışı bırakılan özellikler, projenin ilerleyen sürümleri için doğal bir yol haritası oluşturmaktadır.

3. PİYASA VE REKABET ANALİZİ

3.1. Pazar Büyüklüğü (TAM / SAM / SOM)

Bir ürünün ticari potansiyelini değerlendirebilmek için hedef pazarın büyüklüğünün analiz edilmesi önemlidir. Bu amaçla yaygın olarak kullanılan TAM (Total Addressable Market), SAM (Serviceable Available Market) ve SOM (Serviceable Obtainable Market) yaklaşımı, ürünün teorik olarak ulaşabileceği toplam pazarı, gerçekçi olarak hizmet verebileceği pazarı ve kısa vadede erişebileceği kullanıcı kitlesini ortaya koymaktadır.

TAM (Total Addressable Market)

TAM, görev yönetimi ve proje takip yazılımlarına ihtiyaç duyan tüm potansiyel kullanıcıları kapsamaktadır. Bu pazar; bireysel kullanıcıları, öğrencileri, freelancer çalışanları, startup ekiplerini ve kurumsal işletmeleri içermektedir. Trello, Asana, Notion, Monday.com ve Jira gibi araçların dünya genelinde milyonlarca kullanıcı tarafından tercih edilmesi, görev yönetimi yazılımlarına olan talebin oldukça yüksek olduğunu göstermektedir.

Vanilla JS Kanban projesinin hedeflediği temel problem olan görev planlama ve iş akışı yönetimi, yalnızca yazılım sektöründe değil; eğitim, tasarım, pazarlama, danışmanlık ve kişisel üretkenlik gibi birçok alanda yaygın olarak kullanılmaktadır. Bu nedenle toplam adreslenebilir pazar oldukça geniştir.

SAM (Serviceable Available Market)

SAM, ürünün teknik ve fonksiyonel özellikleri dikkate alındığında gerçekçi olarak hitap ettiği kullanıcı kitlesini ifade etmektedir. Bu proje, özellikle aşağıdaki kullanıcı gruplarına yönelik olarak tasarlanmıştır:

- Üniversite öğrencileri
- Bireysel kullanıcılar
- Freelancer çalışanlar
- 2–5 kişilik küçük ekipler
- Çevrimdışı çalışmaya ihtiyaç duyan kullanıcılar

Ürünün sade yapısı, ücretsiz oluşu ve internet bağlantısı olmadan çalışabilmesi, özellikle karmaşık proje yönetim araçlarını kullanmak istemeyen kullanıcılar için önemli bir avantaj sunmaktadır.

SOM (Serviceable Obtainable Market)

SOM, ürünün kısa ve orta vadede gerçekçi olarak ulaşabileceği kullanıcı kitlesini ifade etmektedir. Bu proje başlangıç aşamasında özellikle aşağıdaki segmentlere odaklanmaktadır:

- Bilgisayar mühendisliği ve diğer üniversite öğrencileri
- Bireysel üretkenlik araçları kullanan kullanıcılar
- Freelancer çalışanlar
- Küçük proje ekipleri

İlk aşamada ürünün GitHub üzerinden açık kaynak olarak paylaşılması, kişisel portföyde sergilenmesi ve sosyal medya aracılığıyla tanıtılması planlanmaktadır. Bu sayede belirli bir niş kullanıcı kitlesine ulaşılması hedeflenmektedir. Ürünün ücretsiz ve kurulumuz çalışması, kullanıcı edinme maliyetini önemli ölçüde düşürmektedir.

Genel Değerlendirme

Yapılan TAM / SAM / SOM analizi sonucunda, geliştirilen Kanban Board uygulamasının geniş bir toplam pazara hitap ettiği, ancak özellikle sade, ücretsiz ve çevrimdışı çalışabilen görev yönetim araçlarına ihtiyaç duyan kullanıcılar için güçlü bir değer önerisi sunduğu görülmektedir. Bu durum, ürünün hem akademik bir proje olarak anlamlı bir probleme çözüm sunduğunu hem de uygun geliştirmelerle gerçek bir ürün haline dönüşme potansiyeline sahip olduğunu göstermektedir.

3.2. Rakip Karşılaştırma Matrisi

3.2 Rakip Karşılaştırma Matrisi

Görev yönetimi ve proje takip yazılımları alanında birçok güçlü rakip bulunmaktadır. Ancak bu araçların büyük bir bölümü ya ücretli özelliklere bağımlı çalışmakta ya da bireysel kullanıcılar için gereğinden fazla karmaşık bir yapı sunmaktadır. Geliştirilen Vanilla JS Kanban uygulaması; sade arayüzü, çevrimdışı çalışabilmesi ve ücretsiz kullanımı ile özellikle bireysel kullanıcılar ve küçük ekipler için farklılaşmaktadır.

Tablo 6: Rakiplerle Özellik Karşılaştırması

Özellik	Bizim Ürün	Rakip 1	Rakip 2	Rakip 3
Ücretsiz plan	✓	✓	✓	✓
Çevrimdışı Çalışma	✓	Sınırlı	Sınırlı	Sınırlı
Kurulum Gerektirmeden Tarayıcıda Çalıştırma	✓	✓	✓	✓
Sürükle Bırak Kanban Yapısı	✓	✓	✓	✓

Checklist Desteği	✓	✓	✓	✓
Son Tarih Belirleme	✓	✓	✓	✓
Kullanıcı Atama	✓	✓	✓	✓
JSON İçe/Dışa Aktarma	✓	Sınırlı	Sınırlı	✓
PDF Raporlama	✓	Sınırlı	Sınırlı	Sınırlı
Keyboard Shortcuts	✓	✓	✓	✓
Dark / Light Tema	✓	✓	✓	✓
Açık Kaynak Olarak Özelleştirilebilir	✓	✗	✗	✓
Sunucu Gerektirmeden Çalışma	✓	✗	✗	✗
Öğrenme Amaçlı Sade Kod Yapısı	✓	✗	✗	✗

Genel Değerlendirme

Rakip analizi sonucunda, Trello ve Notion gibi araçların geniş kullanıcı kitlesine hitap ettiği; ancak çevrimdışı çalışma, tam veri kontrolü ve özelleştirilebilirlik açısından bazı sınırlamalar barındırdığı görülmektedir. Jira ise kurumsal projeler için güçlü özellikler sunmasına rağmen bireysel kullanıcılar için karmaşık bir yapıya sahiptir.

Vanilla JS Kanban uygulaması, özellikle aşağıdaki alanlarda rakiplerinden ayrılmaktadır:

- Tamamen istemci taraflı çalışması
- Kullanıcı verilerinin yalnızca tarayıcı üzerinde saklanması
- İnternet bağlantısı olmadan kullanılabilmesi
- Açık kaynak ve özelleştirilebilir mimariye sahip olması
- Eğitim amaçlı anlaşılır kod yapısı sunması
- Ücretsiz ve kurulumuz kullanım imkânı sağlaması

Bu özellikler, ürünü özellikle öğrenciler, bireysel kullanıcılar ve küçük ekipler için güçlü bir alternatif haline getirmektedir.

3.3. Detaylı Rakip İncelemesi

Bu bölümde, görev yönetimi ve proje takip alanında yaygın olarak kullanılan üç önemli rakip ürün detaylı olarak incelenmiştir. Her ürün; hedef kullanıcı kitlesi, fiyatlandırma modeli, güçlü yönleri ve sınırlılıkları açısından değerlendirilmiş, geliştirilen Vanilla JS Kanban uygulamasının bu ürünlere karşı konumu ortaya konmuştur.

1. Trello

Kuruluş Yılı: 2011

Geliştirici Şirket: [Atlassian](#)

Resmî Web Sitesi: [Trello](#)

Kısa Tanım:

Trello, Kanban tabanlı görev yönetimi araçları arasında en yaygın kullanılan çözümlerden biridir. Kart ve kolon yapısı sayesinde kullanıcıların görevlerini görsel olarak organize etmesine olanak tanımaktadır.

Güçlü Yönleri:

- Kullanımı oldukça kolay ve sezgiseldir.
- Geniş eklenti (Power-Up) ekosistemine sahiptir.
- Web, masaüstü ve mobil platform desteği sunar.

Zayıf Yönleri:

- Gelişmiş özelliklerin bir kısmı ücretli planlara bağlıdır.
- Sürekli internet bağlantısına bağımlıdır.
- Veriler üçüncü taraf sunucularda saklanmaktadır.

2. Notion

Kuruluş Yılı: 2016

Geliştirici Şirket: [Notion](#)

Resmî Web Sitesi: [Notion](#)

Kısa Tanım:

Notion, not alma, bilgi yönetimi ve proje takibini tek bir platformda birleştiren çok yönlü bir üretkenlik aracıdır.

Güçlü Yönleri:

- Son derece esnek veri yapıları sunar.
- Dokümantasyon ve proje yönetimini birleştirir.
- Güçlü şablon desteğine sahiptir.

Zayıf Yönleri:

- Yeni kullanıcılar için öğrenme eğrisi yüksektir.
- Yoğun kullanımda performans düşüşleri görülebilir.
- Basit görev takibi için gereğinden karmaşık olabilir.

3. Jira

Kuruluş Yılı: 2002

Geliştirici Şirket: [Atlassian](#)

Resmî Web Sitesi: [Jira](#)

Kısa Tanım:

Jira, özellikle yazılım geliştirme ekipleri için tasarlanmış, gelişmiş proje ve iş takip aracıdır.

Güçlü Yönleri:

- Scrum ve Agile süreçlerine güçlü destek sağlar.
- Detaylı raporlama ve iş akışı özelleştirmeleri sunar.
- Kurumsal ekipler için kapsamlı entegrasyonlar içerir.

Zayıf Yönleri:

- Arayüzü karmaşık olabilir.
- Küçük ekipler ve bireysel kullanıcılar için fazla ağırdır.
- Kurulum ve yapılandırma süreci zaman alabilir.

Vanilla JS Kanban'ın Konumu

Geliştirilen Vanilla JS Kanban uygulaması, yukarıdaki rakiplerle karşılaştırıldığında aşağıdaki temel avantajlara sahiptir:

- Tamamen ücretsizdir.
- İnternet bağlantısı olmadan çalışabilir.
- Veriler yalnızca kullanıcı tarayıcısında saklanır.
- Açık kaynak yapısı sayesinde özelleştirilebilir.
- Framework bağımlılığı olmadan sade bir mimariye sahiptir.

Buna karşılık, ürünün mevcut sürümünde:

- Mobil uygulama desteği bulunmamaktadır.
- Gerçek zamanlı çoklu kullanıcı senkronizasyonu yoktur.
- Bulut tabanlı veri yedekleme desteği sunulmamaktadır.

Sonuç

Detaylı rakip incelemesi, Trello'nun kullanım kolaylığı, Notion'un esnekliği ve Jira'nın kurumsal gücüyle öne çıktığını göstermektedir. Vanilla JS Kanban ise özellikle sade, hızlı, çevrimdışı çalışabilen ve kullanıcı verisini tamamen yerel olarak saklayan yapısıyla farklılaşmaktadır. Bu özellikler, ürünü öğrenciler, bireysel kullanıcılar ve küçük ekipler için güçlü bir alternatif haline getirmektedir.

3.4. SWOT Analizi

SWOT analizi, bir ürünün güçlü yönlerini (Strengths), zayıf yönlerini (Weaknesses), fırsatlarını (Opportunities) ve karşılaşılabileceği tehditleri (Threats) sistematik olarak değerlendirmek için kullanılan stratejik bir analiz yöntemidir. Geliştirilen Vanilla JS Kanban uygulaması için hazırlanan SWOT matrisi Tablo 7'de sunulmuştur.

Tablo 7: SWOT Matrisi

Güçlü Yönler (S)	Zayıf Yönler (W)
Tamamen ücretsiz ve açık kaynak yapıya sahiptir.	Sunucu tabanlı veri senkronizasyonu bulunmamaktadır.
Çevrimdışı çalışabilmektedir.	Gerçek zamanlı çoklu kullanıcı desteği yoktur.
Framework bağımlılığı olmadan geliştirilmiştir.	Mobil uygulama sürümü bulunmamaktadır.
Kullanıcı verileri yalnızca yerel tarayıcıda saklanmaktadır.	Tarayıcı verileri silinirse kullanıcı verileri kaybolabilir.
Modüler ve genişletilebilir mimariye sahiptir.	Büyük ölçekli ekipler için sınırlı özellik sunmaktadır.
Sade ve kullanıcı dostu arayüz sunmaktadır.	Kimlik doğrulama ve yetkilendirme mekanizması bulunmamaktadır.
Fırsatlar (Opportunities)	Tehditler (Threats)
Açık kaynak topluluğu tarafından geliştirilebilir.	Trello, Notion ve Jira gibi güçlü rakipler bulunmaktadır.

Bulut senkronizasyonu eklenerek SaaS ürüne dönüştürülebilir.	Büyük platformlar ücretsiz planlarını genişletebilir.
Eğitim amaçlı kaynak ve portföy projesi olarak değerlendirilebilir.	Yeni kullanıcıların mevcut araçlardan geçiş yapma isteği düşük olabilir.
Üniversite öğrencileri ve freelancer kullanıcılar için niş bir çözüm sunabilir.	Kullanıcıların verilerini yalnızca tarayıcıda saklamaya güvenmemesi mümkündür.
Mobil uygulama desteği ile kullanıcı kitlesi artırılabilir.	Tarayıcı teknolojilerindeki değişiklikler uyumluluk sorunları oluşturabilir.

Genel Değerlendirme

SWOT analizi sonucunda, Vanilla JS Kanban uygulamasının en güçlü yönlerinin çevrimdışı çalışabilmesi, açık kaynak olması, sade yapısı ve kullanıcı verilerini tamamen yerel olarak saklaması olduğu görülmektedir. Bu özellikler, özellikle bireysel kullanıcılar ve küçük ekipler için önemli avantajlar sunmaktadır.

Bununla birlikte, bulut tabanlı senkronizasyon, gerçek zamanlı iş birliği ve kimlik doğrulama gibi modern SaaS özelliklerinin bulunmaması ürünün mevcut sınırlılıkları arasında yer almaktadır. Ancak modüler mimari sayesinde bu özelliklerin gelecekte eklenmesi mümkündür.

Genel olarak değerlendirildiğinde, proje güçlü bir teknik temel üzerine kurulmuş olup uygun geliştirmelerle gerçek bir ürün haline dönüşme potansiyeline sahiptir.

3.5. Positioning Statement (Konumlandırma)

Aşağıda geliştirilen Vanilla JS Kanban uygulaması için hazırlanan konumlandırma ifadesi yer almaktadır.

For bireysel kullanıcılar, üniversite öğrencileri ve küçük ekipler

Who görevlerini sade, hızlı ve çevrimdışı çalışabilen bir sistem üzerinden yönetmek istemektedir

Our product is a web tabanlı Kanban Board uygulaması

That görevlerin görsel olarak organize edilmesini, detaylı olarak yönetilmesini ve tarayıcı üzerinde güvenli biçimde saklanmasını sağlar

Unlike Trello, Notion ve Jira gibi internet bağlantısına ve merkezi sunuculara bağımlı proje yönetim araçları

Our product tamamen istemci taraflı çalışır, kullanıcı verilerini yerel olarak saklar, çevrimdışı kullanılabilir ve açık kaynak yapısı sayesinde kolayca özelleştirilebilir.

Türkçe Açıklama

Vanilla JS Kanban, özellikle karmaşık ve internet bağımlı proje yönetim araçları yerine daha sade, hızlı ve kurulumuz çalışan bir çözüm arayan kullanıcılar için geliştirilmiştir. Ürünün temel farklılaştırıcı özellikleri şunlardır:

- Tamamen tarayıcı üzerinde çalışması
- İnternet bağlantısı olmadan kullanılabilmesi
- Kullanıcı verilerinin yalnızca yerel cihazda saklanması
- Açık kaynak ve özelleştirilebilir mimari sunması
- Framework bağımlılığı olmadan geliştirilmiş olması

Bu konumlandırma, ürünün özellikle öğrenciler, bireysel kullanıcılar ve küçük ekipler için güçlü bir alternatif olduğunu göstermektedir.

4. TEKNOLOJİ YIĞINI (TECH STACK)

4.1. Katmanlar — Özet Tablosu

4.1 Katmanlar — Özet Tablosu

Bu bölümde, proje kapsamında kullanılan temel teknolojiler katman bazında özetlenmektedir. Geliştirilen Vanilla JS Kanban uygulaması, tamamen istemci taraflı çalışan bir web uygulamasıdır. Uygulamanın kullanıcı arayüzü, iş mantığı, veri saklama ve raporlama işlemleri farklı teknolojiler kullanılarak modüler bir şekilde gerçekleştirilmiştir.

Tablo 8: Projede Kullanılan Teknoloji Katmanları

Katman	Teknolojiler
Frontend	HTML5, CSS3, Vanilla JavaScript (ES6+)
Kullanıcı Arayüzü	CSS Grid, Flexbox, Custom Modal Components
State Yönetimi	Merkezi <code>App.data</code> ve <code>App.users</code> yapısı
Sürükle-Bırak	HTML5 Drag & Drop API
Veri Kalıcılığı	localStorage
Veri Aktarımı	JSON Import / Export
PDF Raporlama	JsPDF
Tema Yönetimi	CSS Variables + localStorage
Klavye Kısayolları	JavaScript <code>keydown</code> event listeners
Tarayıcı API'leri	DOM API, Web Storage API, File API
Test ve Doğrulama	Manuel fonksiyonel testler
Versiyon Kontrol	GitHub
Çalışma Ortamı	Modern web tarayıcıları (Chrome, Firefox, Edge, Safari)

Bu teknoloji yığını, herhangi bir backend altyapısına ihtiyaç duymadan, tamamen tarayıcı üzerinde çalışan ve kullanıcı verilerini yerel olarak saklayan bir görev yönetim sistemi geliştirilmesini mümkün kılmıştır. Ayrıca kullanılan teknolojilerin büyük bölümü modern web standartlarına dayandığı için uygulama taşınabilir, anlaşılabilir ve genişletilebilir bir yapıya sahiptir.

Sonraki alt bölümlerde, tabloda yer alan temel teknolojiler ayrı ayrı ele alınacak ve her bir teknolojinin projedeki rolü, seçilme gerekçeleri ve alternatif çözümlerle karşılaştırmaları detaylı olarak açıklanacaktır.

4.2. Frontend

4.2.1. Vanilla JavaScript (ES2024)

Vanilla JavaScript, herhangi bir framework veya ek kütüphane kullanılmadan doğrudan tarayıcının sunduğu JavaScript dil özellikleri ile uygulama geliştirilmesini ifade etmektedir. Bu projede uygulamanın temel iş mantığı, kullanıcı etkileşimleri, veri yönetimi ve dinamik arayüz güncellemeleri tamamen Vanilla JavaScript kullanılarak gerçekleştirilmiştir.

Neden Seçildi?

Vanilla JavaScript'in tercih edilmesinin temel nedenleri aşağıda özetlenmiştir:

1. Framework bağımlılığı oluşturmada temel web teknolojilerinin etkin kullanımını göstermek.
2. Uygulamanın çalışma mantığını daha şeffaf ve anlaşılır hale getirmek.
3. Ek bağımlılıkları azaltarak daha hafif ve hızlı bir yapı elde etmek.
4. Eğitim amacıyla JavaScript dilinin çekirdek özelliklerini doğrudan uygulamak.

Temel Özellikler

- ES6+ sınıf ve modül yapıları
- Arrow function kullanımı
- Template literals
- Destructuring assignment
- Array methods (map, filter, find)
- Event-driven programlama
- DOM manipülasyonu
- JSON işlemleri

Alternatifler ve Neden Seçilmedi?

Tablo 9: Proje İçin Farklı Alternatifler ve Seçilmeme Nedenleri

Alternatif	Seçilmeme Nedeni
React	Proje ölçeği için gereksiz ek yapı ve bağımlılık oluşturur.
Vue.js	Öğrenme eğrisi ve framework bağımlılığı yaratır.
Angular	Küçük ölçekli proje için fazla kapsamlıdır.

jQuery	Modern JavaScript özellikleri karşısında ek faydası sınırlıdır.
---------------	---

Trade-Off'lar

Avantajlar:

- Hafif ve hızlıdır.
- Ek bağımlılık gerektirmez.
- Kodun çalışma mantığı daha net görülür.
- Eğitim amaçlı oldukça uygundur.

Dezavantajlar:

- Büyük projelerde yapı yönetimi zorlaşabilir.
- Hazır component ekosistemi bulunmaz.
- Geliştiricinin daha fazla kod yazması gerekir.

Öğrenme Kaynakları

- [MDN Web Docs – JavaScript Guide](#)
- [JavaScript.info](#)
- [ECMAScript Language Specification](#)

Projedeki Rolü

Vanilla JavaScript, uygulamanın temel omurgasını oluşturmaktadır. Kolon ve kart işlemleri, sürükle-bırak mekanizması, localStorage üzerinden veri yönetimi, tema değişimi, klavye kısayolları, JSON/PDF export işlemleri ve kullanıcı arayüzünün dinamik olarak oluşturulması bu teknoloji kullanılarak gerçekleştirilmiştir. Bu sayede proje, herhangi bir framework bağımlılığı olmadan modern web uygulama geliştirme prensiplerini uygulamalı olarak ortaya koymaktadır.

4.2.2. Web Components (HTML5)

HTML5, web sayfalarının yapısal iskeletini oluşturan standart işaretleme dilinin güncel sürümüdür. Bu projede uygulamanın tüm sayfa yapısı, form bileşenleri, modal pencereleri ve Kanban panosunu oluşturan temel DOM elemanları HTML5 kullanılarak oluşturulmuştur.

Neden Seçildi?

HTML5'in tercih edilmesinin temel nedenleri aşağıda verilmiştir:

1. Web uygulamalarının temel yapısal katmanını oluşturmaları.
2. Semantik etiketler sayesinde okunabilir ve erişilebilir bir yapı sunması.

3. Modern tarayıcılar tarafından doğal olarak desteklenmesi.
4. Ek araç veya derleme süreci gerektirmemesi.

Temel Özellikler

- Semantik etiketler (header, main, section, article, footer)
- Form elemanları (input, textarea, select, button)
- Veri taşıma için data-* attribute'ları
- Modal yapılar için esnek DOM düzeni
- Dosya seçimi için input type="file"
- Tarih seçimi için input type="date"

Alternatifler ve Neden Seçilmedi?

Tablo 10: Proje İçin Farklı Alternatifler ve Seçilmeme Nedenleri

Alternatif	Seçilmeme Nedeni
JSX	React gibi framework'lere bağımlıdır.
Template engine'ler	Bu proje için gereksiz ek yapı oluşturur.
Web Components	Proje ölçeği için ek soyutlama gerektirir.

Trade-Off'lar

Avantajlar:

- Tüm tarayıcılarda doğal olarak desteklenir.
- Hafif ve standart bir çözümdür.
- Semantik yapı sayesinde erişilebilirliği artırır.

Dezavantajlar:

- Dinamik davranış için JavaScript desteği gerekir.
- Bileşen tabanlı yapı sunmaz.

Öğrenme Kaynakları

- [MDN Web Docs – HTML](#)
- [WHATWG HTML Standard](#)

Projedeki Rolü

HTML5, uygulamanın temel yapısını oluşturmaktadır. Kanban kolonları, kart bileşenleri, modal pencereleri, kullanıcı formları, filtreleme alanları ve dosya yükleme bileşenleri HTML5 ile tanımlanmıştır. JavaScript tarafından dinamik olarak oluşturulan içerikler de HTML tabanlı DOM yapısı üzerinde çalışmaktadır.

4.2.3. Tailwind CSS 3

CSS3, web uygulamalarının görsel tasarımını ve kullanıcı arayüzünün stil özelliklerini belirleyen standart stil dilidir. Bu projede uygulamanın renk paleti, tipografi, yerleşim düzeni, animasyonlar, tema yönetimi ve responsive tasarım özellikleri CSS3 kullanılarak geliştirilmiştir.

Neden Seçildi?

CSS3'ün tercih edilmesinin temel nedenleri aşağıda verilmiştir:

1. Uygulamanın görsel tasarımının tamamen özelleştirilebilmesini sağlaması.
2. Dark/Light tema desteğinin CSS değişkenleri ile kolayca uygulanabilmesi.
3. Responsive tasarım için Flexbox ve Grid gibi güçlü layout sistemleri sunması.
4. Herhangi bir CSS framework'üne ihtiyaç duymadan hafif ve esnek bir yapı sağlaması.

Temel Özellikler

- CSS Variables (--primary-color, --background-color vb.)
- Flexbox
- CSS Grid
- Media Queries
- Transition ve Animation
- Hover ve Focus efektleri
- Glassmorphism tasarım yaklaşımı
- Dark / Light tema desteği

Alternatifler ve Neden Seçilmedi?

Tablo 11: Proje İçin Farklı Alternatifler ve Seçilmeme Nedenleri

Alternatif	Seçilmeme Nedeni
Bootstrap	Hazır bileşen yapısı tasarım esnekliğini sınırlandırabilir.
Tailwind CSS	Utility-first yaklaşımı proje için gerekli görülmemiştir.
Material UI	Framework bağımlılığı oluşturur.

Trade-Off'lar

Avantajlar:

- Tam kontrol ve yüksek özelleştirilebilirlik sağlar.
- Ek bağımlılık gerektirmez.

- Hafif ve performanslıdır.

Dezavantajlar:

- Hazır bileşen kütüphanelerine göre daha fazla manuel stil yazımı gerektirir.
- Büyük projelerde stil organizasyonu zorlaşabilir.

Öğrenme Kaynakları

- [MDN Web Docs – CSS](#)
- [CSS-Tricks](#)

Projedeki Rolü

CSS3, uygulamanın tüm görsel katmanını oluşturmaktadır. Kanban kolonlarının düzeni, kart tasarımları, modal pencereler, kullanıcı avatarları, arama ve filtreleme bileşenleri, tema değişimi ve responsive tasarım özellikleri CSS3 ile gerçekleştirilmiştir. Ayrıca geçiş animasyonları ve görsel geri bildirimler sayesinde kullanıcı deneyimi önemli ölçüde iyileştirilmiştir.

4.3. State & Drag-Drop

4.3.1. State Yönetimi (Merkezi App.data ve App.users Yapısı)

State yönetimi, uygulamanın mevcut durumunu temsil eden verilerin merkezi bir yapıda tutulması ve bu verilerde meydana gelen değişikliklerin kullanıcı arayüzüne yansıtılması sürecidir. Bu projede herhangi bir harici state management kütüphanesi kullanılmadan, uygulamanın tüm verileri `App.data` ve `App.users` isimli iki temel yapı içerisinde tutulmuştur.

Neden Seçildi?

Bu yaklaşımın tercih edilmesinin temel nedenleri aşağıda verilmiştir:

1. Uygulama verilerinin tek bir merkezden yönetilmesini sağlamak.
2. Veri güncellemeleri ile kullanıcı arayüzü arasındaki senkronizasyonu kolaylaştırmak.
3. Harici kütüphane kullanmadan sade ve anlaşılır bir mimari kurmak.
4. Kodun bakımını ve hata ayıklamayı kolaylaştırmak.

Temel Özellikler

- `App.data`: Kolonlar ve kartlar gibi ana Kanban verilerini tutar.
- `App.users`: Kullanıcı listesini saklar.

- Merkezi veri yapısı sayesinde tek kaynak (single source of truth) yaklaşımı uygulanır.
- Her veri güncellemesinden sonra arayüz yeniden render edilir.
- localStorage ile senkron çalışır.
- Modüler yapı tarafından ortak olarak kullanılır.

Alternatifler ve Neden Seçilmedi?

Tablo 12: Proje İçin Farklı Alternatifler ve Seçilmeme Nedenleri

Alternatif	Seçilmeme Nedeni
Redux	Proje ölçeği için gereğinden karmaşıktır.
MobX	Ek bağımlılık gerektirir.
Vuex / Pinia	Vue.js ekosistemine bağlıdır.
Context API	React tabanlı projelerde kullanılır.

Trade-Off'lar

Avantajlar:

- Basit ve anlaşılırdır.
- Harici bağımlılık gerektirmez.
- Tüm veriler merkezi olarak yönetilir.
- Hata ayıklama kolaylaşır.

Dezavantajlar:

- Çok büyük projelerde yapı karmaşık hale gelebilir.
- İnce ayarlı reaktif güncellemeler sunmaz.
- Gerektiğinde tüm arayüz yeniden render edilebilir.

Öğrenme Kaynakları

- [MDN Web Docs – JavaScript Objects](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects)
- [JavaScript.info – Objects](https://javascript.info/objects)

Projedeki Rolü

Uygulamadaki tüm kolonlar, kartlar, checklist öğeleri ve kullanıcı bilgileri bu merkezi state yapısı içerisinde tutulmaktadır. Kart ekleme, düzenleme, taşıma, silme ve kullanıcı işlemleri önce state üzerinde gerçekleştirilmekte, ardından veriler localStorage'a kaydedilmekte ve kullanıcı arayüzü yeniden oluşturulmaktadır. Bu yaklaşım, uygulamanın veri tutarlılığını koruyarak modüler mimarinin temelini oluşturmaktadır.

4.3.2. HTML5 Drag & Drop API

HTML5 Drag & Drop API, kullanıcıların DOM elemanlarını sürükleyip bırakmasına olanak tanıyan yerleşik tarayıcı API'sidir. Bu projede görev kartlarının ve kolonların farklı konumlara taşınabilmesi için bu API kullanılmıştır. Böylece kullanıcılar, görev durumlarını doğal ve sezgisel bir etkileşim yöntemiyle güncelleyebilmektedir.

Neden Seçildi?

HTML5 Drag & Drop API'nin tercih edilmesinin temel nedenleri aşağıda verilmiştir:

1. Modern tarayıcılar tarafından yerleşik olarak desteklenmesi.
2. Harici bir kütüphane gerektirmemesi.
3. Kullanıcı deneyimini önemli ölçüde iyileştirmesi.
4. Kartların ve kolonların görsel olarak yeniden düzenlenmesini sağlaması.

Temel Özellikler

- dragstart, dragover, drop, dragend olayları
- DOM elemanlarının dinamik olarak taşınması
- dataTransfer nesnesi ile veri aktarımı
- Placeholder kullanımı ile görsel geri bildirim
- Aynı kolon içinde veya farklı kolonlar arasında taşıma
- Kolonların yeniden sıralanması

Alternatifler ve Neden Seçilmedi?

Tablo 13: Proje İçin Farklı Alternatifler ve Seçilmeme Nedenleri

Alternatif	Seçilmeme Nedeni
SortableJS	Harici bağımlılık ekler; proje, mümkün olduğunca saf web API'leri ile geliştirilmiştir.
Dragula	Ek kütüphane gerektirir.
Interact.js	Proje ölçeği için fazla kapsamlıdır.

Trade-Off'lar

Avantajlar:

- Yerleşik tarayıcı desteği sunar.
- Ek kütüphane gerektirmez.
- Hafif ve performanslıdır.
- Öğrenme açısından temel web API'lerini kullanma fırsatı sağlar.

Dezavantajlar:

- Mobil dokunmatik desteği sınırlıdır.
- Bazı tarayıcı farklılıkları ek kod gerektirebilir.
- Gelişmiş davranışların manuel olarak uygulanması gerekir.

Öğrenme Kaynakları

- [MDN Web Docs – HTML Drag and Drop API](#)
- [JavaScript.info – Drag and Drop](#)

Projedeki Rolü

Kanban panosundaki kartlar ve kolonlar `draggable` özelliği ile sürüklenabilir hale getirilmiştir. Kullanıcı bir kartı farklı bir kolona bıraktığında, DOM üzerindeki yeni konum okunmakta ve merkezi state yapısı güncellenmektedir. Ardından veriler `localStorage`'a kaydedilmekte ve kullanıcı arayüzü yeni düzene göre yeniden render edilmektedir. Bu mekanizma, Kanban metodolojisinin en temel etkileşimini oluşturmaktadır.

4.4. Depolama**4.4.1. JSON Import / Export**

JSON Import / Export mekanizması, uygulama verilerinin taşınabilir ve yedeklenebilir hale getirilmesini sağlayan veri aktarım yöntemidir. Bu projede kullanıcılar, mevcut Kanban panosunu JSON dosyası olarak dışa aktarabilmekte ve daha sonra aynı dosyayı tekrar içe aktararak verilerini geri yükleyebilmektedir. Bu özellik, tarayıcı verilerinin silinmesi durumunda veri kaybını önlemek ve farklı cihazlar arasında veri taşımak amacıyla geliştirilmiştir.

Neden Seçildi?

JSON tabanlı veri aktarımının tercih edilme nedenleri aşağıda verilmiştir:

1. İnsan tarafından okunabilir ve standart bir veri formatı olması.
2. JavaScript nesneleri ile doğal uyum göstermesi.
3. Yedekleme ve geri yükleme işlemlerini kolaylaştırması.
4. Kullanıcı verilerinin taşınabilirliğini sağlaması.

Temel Özellikler

- Tüm pano verilerinin tek dosyada saklanması
- İnsan tarafından okunabilir yapı
- `JSON.stringify()` ile dışa aktarma

- `JSON.parse()` ile içe aktarma
- Dosya indirme ve dosya yükleme desteği
- Veri doğrulama ve hata kontrolü

Alternatifler ve Neden Seçilmedi?

Tablo 14: Proje İçin Farklı Alternatifler ve Seçilmeme Nedenleri

Alternatif	Seçilmeme Nedeni
CSV	Hiyerarşik veri yapıları için uygun değildir.
XML	JSON'a göre daha karmaşıktır.
PDF	Düzenlenebilir veri formatı değildir.
Sunucu tabanlı yedekleme	Ek backend altyapısı gerektirir.

Trade-Off'lar

Avantajlar:

- Basit ve standart bir formattır.
- Veriler kolayca yedeklenebilir.
- Farklı cihazlara aktarım yapılabilir.
- Hata durumlarında manuel inceleme mümkündür.

Dezavantajlar:

- Kullanıcının dosya yönetimini manuel yapması gerekir.
- Dosya bozulursa içe aktarma başarısız olabilir.
- Otomatik bulut senkronizasyonu sağlamaz.

Öğrenme Kaynakları

- [MDN Web Docs – JSON](#)
- [MDN Web Docs – File API](#)

Projedeki Rolü

Kullanıcı, uygulama içerisindeki export özelliğini kullanarak tüm pano verilerini biçimlendirilmiş bir JSON dosyası olarak indirebilmektedir. Daha sonra bu dosya import özelliği ile tekrar sisteme yüklenerek kolonlar, kartlar, checklist öğeleri ve kullanıcı bilgileri geri getirilebilmektedir. Bu mekanizma, veri yedekleme ve taşınabilirlik açısından önemli bir güvenlik katmanı oluşturmaktadır.

4.4.2. LocalStorage (preferences)

localStorage, Web Storage API'nin bir parçası olan ve verilerin tarayıcı üzerinde anahtar-değer (key-value) formatında kalıcı olarak saklanmasını sağlayan yerleşik bir mekanizmadır. Bu projede kolonlar, görev kartları, kullanıcı bilgileri, tema tercihleri ve filtre ayarları gibi verilerin saklanması için localStorage kullanılmıştır. Böylece kullanıcı verileri sayfa yenilense veya tarayıcı kapatılıp tekrar açılrsa dahi korunmaktadır.

Neden Seçildi?

localStorage kullanımının temel nedenleri aşağıda verilmiştir:

1. Herhangi bir sunucu veya veritabanı altyapısına ihtiyaç duymadan veri kalıcılığı sağlaması.
2. Modern tarayıcılar tarafından yerleşik olarak desteklenmesi.
3. Kullanımının oldukça basit ve hızlı olması.
4. Projenin tamamen istemci taraflı çalışma hedefiyle uyumlu olması.

Temel Özellikler

- Anahtar-değer (key-value) yapısında veri saklama
- Verilerin tarayıcı kapatılrsa dahi korunması
- `setItem()`, `getItem()`, `removeItem()` ve `clear()` metodları
- JSON formatında karmaşık veri yapılarının saklanabilmesi
- Yaklaşık 5–10 MB depolama kapasitesi
- Aynı origin altında erişim

Alternatifler ve Neden Seçilmedi?

Tablo 15: Proje İçin Farklı Alternatifler ve Seçilmeme Nedenleri

Alternatif	Seçilmeme Nedeni
IndexedDB	Daha güçlü olmasına rağmen proje ölçeği için gereğinden karmaşıktır.
Session Storage	Tarayıcı sekmesi kapatıldığında veriler silinir.
Sunucu tabanlı veritabanı	Ek backend altyapısı gerektirir.
Cookies	Depolama kapasitesi sınırlıdır ve bu amaç için uygun değildir.

Trade-Off'lar

Avantajlar:

- Basit ve hızlıdır.
- Sunucu gerektirmez.
- Çevrimdışı kullanım için uygundur.
- JSON veriler kolayca saklanabilir.

Dezavantajlar:

- Depolama kapasitesi sınırlıdır.
- Büyük veri kümeleri için uygun değildir.
- Veriler tarayıcı temizlenirse kaybolabilir.
- Yapısal sorgulama yeteneği yoktur.

Öğrenme Kaynakları

- [MDN Web Docs – Window.localStorage](#)
- [Web Storage API](#)

Projedeki Rolü

Uygulamanın ana veri yapısı olan kolonlar, kartlar ve kullanıcı bilgileri localStorage üzerinde JSON formatında saklanmaktadır. Kullanıcı bir işlem gerçekleştirdiğinde veriler güncellenmekte ve `localStorage.setItem()` ile kaydedilmektedir. Uygulama açıldığında ise kayıtlı veriler `getItem()` ile okunmakta ve sistem son durumuyla yüklenmektedir. Ayrıca tema tercihleri ve filtre ayarları da localStorage üzerinde tutulmaktadır. Bu yapı, projenin çevrimdışı çalışabilmesini ve kullanıcı verilerinin kalıcı olmasını sağlamaktadır.

4.5. PWA

4.5.1. jsPDF (Workbox)

jsPDF, tarayıcı ortamında JavaScript kullanarak PDF dosyaları oluşturmayı sağlayan açık kaynaklı bir kütüphanedir. Bu projede mevcut Kanban panosundaki görevlerin düzenli bir rapor halinde PDF formatında dışa aktarılabilmesi için jsPDF kullanılmıştır. Böylece kullanıcılar görev durumlarını arşivleyebilmekte, paylaşabilmekte ve çıktı alabilmektedir.

Neden Seçildi?

jsPDF kütüphanesinin tercih edilmesinin temel nedenleri aşağıda verilmiştir:

1. Tarayıcı üzerinde doğrudan PDF oluşturabilmesi.
2. Sunucu tarafında ek işlem gerektirmemesi.
3. JavaScript ile kolay entegre edilebilmesi.
4. Açık kaynak ve yaygın olarak kullanılan bir çözüm olması.

Temel Özellikler

- Metin, tablo ve temel grafik öğeleri ekleyebilme
- Dosya indirme işlemini tarayıcı üzerinden gerçekleştirme
- Sayfa boyutu ve yönlendirme desteği
- Çok sayfalı PDF üretimi
- Unicode karakter desteği (ek dönüştürme ile)
- Tamamen istemci taraflı çalışma

Alternatifler ve Neden Seçilmedi?

Tablo 16: Proje İçin Farklı Alternatifler ve Seçilmeme Nedenleri

Alternatif	Seçilmeme Nedeni
PDFMake	Daha kapsamlı olmakla birlikte proje ihtiyaçları için gereğinden fazla özellik sunmaktadır.
html2pdf.js	HTML çıktısına bağımlı çalışmaktadır.
Sunucu taraflı PDF üretimi	Ek backend altyapısı gerektirir.

Trade-Off'lar

Avantajlar:

- Hızlı ve kolay entegrasyon sağlar.
- Sunucu gerektirmez.
- Kullanıcı verilerinin cihaz dışına çıkmasını engeller.
- Tarayıcı üzerinde doğrudan çalışır.

Dezavantajlar:

- Karmaşık rapor düzenleri manuel olarak oluşturulmalıdır.
- Türkçe karakter desteği ek işleme ihtiyaç duyabilir.
- Büyük raporlarda sayfa düzeni yönetimi zorlaşabilir.

Öğrenme Kaynakları

- [jsPDF Resmî Dokümantasyonu](#)
- [jsPDF Examples](#)

Projedeki Rolü

Kullanıcı, PDF export özelliğini çalıştırdığında mevcut Kanban panosundaki kolonlar ve kartlar okunarak düzenli bir rapor oluşturulmaktadır. Görev başlıkları, açıklamalar, son tarihler ve durum bilgileri PDF içerisine eklenmekte ve dosya tarayıcı üzerinden indirilmektedir. Bu özellik, kullanıcıların proje durumlarını dokümanete etmesini ve paylaşmasını kolaylaştırmaktadır.

4.5.2. Web App Manifest (File API)

File API, tarayıcı ortamında kullanıcı tarafından seçilen dosyaların okunmasını ve işlenmesini sağlayan yerleşik web API'sidir. Bu projede JSON formatındaki yedek dosyalarının içe aktarılması için File API kullanılmıştır. Böylece kullanıcılar daha önce dışa aktardıkları pano verilerini seçerek uygulama içerisine yeniden yükleyebilmektedir.

Neden Seçildi?

File API'nin tercih edilmesinin temel nedenleri aşağıda verilmiştir:

1. Modern tarayıcılar tarafından yerleşik olarak desteklenmesi.
2. Kullanıcının yerel dosyalarını güvenli bir şekilde okuyabilmesi.
3. JSON import özelliğinin istemci tarafında gerçekleştirilmesini sağlaması.
4. Harici kütüphane gerektirmemesi.

Temel Özellikler

- `input type="file"` desteği
- `FileReader` ile dosya içeriği okuma
- Metin tabanlı dosyaların işlenmesi
- İstemci tarafında doğrulama yapılabilmesi
- JSON verilerinin ayrıştırılması

Alternatifler ve Neden Seçilmedi?

Tablo 17: Proje İçin Farklı Alternatifler ve Seçilmeme Nedenleri

Alternatif	Seçilmeme Nedeni
Sunucu üzerinden dosya yükleme	Backend altyapısı gerektirir.
Drag & Drop dosya yükleme	Mevcut ihtiyaç için ek karmaşıklık oluşturur.
IndexedDB	Yedekleme amacı için doğrudan uygun

senkronizasyonu	değildir.
-----------------	-----------

Trade-Off'lar

Avantajlar:

- Tarayıcıda doğal olarak desteklenir.
- Sunucu gerektirmez.
- Kullanıcı verilerinin cihaz dışına çıkmasını önler.
- Kullanımı basit ve güvenlidir.

Dezavantajlar:

- Kullanıcı dosya seçimini manuel olarak yapmalıdır.
- Hatalı dosyalar için doğrulama gerektirir.
- Büyük dosyalarda ek hata kontrolü gerekebilir.

Öğrenme Kaynakları

- [MDN Web Docs – File API](#)
- [MDN Web Docs – FileReader](#)

Projedeki Rolü

Kullanıcı JSON içe aktarma işlemini başlattığında bir dosya seçme penceresi açılmaktadır. Seçilen dosya `FileReader` aracılığıyla okunmakta, JSON formatı doğrulanmakta ve veriler uygulamanın merkezi state yapısına aktarılmaktadır. Ardından veriler `localStorage`'a kaydedilmekte ve kullanıcı arayüzü yeniden oluşturulmaktadır. Bu mekanizma, veri yedeklerinin güvenli bir şekilde geri yüklenmesini sağlamaktadır.

4.6. Hosting

4.6.1. Modern Web Tarayıcıları

Modern web tarayıcıları, geliştirilen uygulamanın çalıştığı temel çalışma ortamını oluşturmaktadır. Vanilla JS Kanban uygulaması herhangi bir kurulum veya sunucu bağımlılığı olmadan doğrudan tarayıcı üzerinde çalışacak şekilde tasarlanmıştır. Bu nedenle Chrome, Firefox, Edge ve Safari gibi güncel tarayıcılar, uygulamanın doğal çalışma platformu olarak değerlendirilmiştir.

Neden Seçildi?

Modern web tarayıcılarının çalışma ortamı olarak tercih edilmesinin temel nedenleri aşağıda verilmiştir:

1. Kullanıcıların ek bir kurulum yapmadan uygulamayı çalıştırabilmesini sağlaması.
2. HTML, CSS ve JavaScript standartlarını yerleşik olarak desteklemesi.
3. localStorage, File API ve Drag & Drop API gibi gerekli web API'lerini sunması.
4. Platform bağımsız kullanım imkânı sağlaması.

Temel Özellikler

- HTML5, CSS3 ve JavaScript desteği
- DOM API
- Web Storage API
- File API
- Drag & Drop API
- Yazdırma ve PDF indirme desteği
- Responsive tasarım desteği

Alternatifler ve Neden Seçilmedi?

Tablo 18: Proje İçin Farklı Alternatifler ve Seçilmeme Nedenleri

Alternatif	Seçilmeme Nedeni
Masaüstü uygulama (Electron)	Ek paketleme ve dağıtım süreci gerektirir.
Mobil uygulama	Ayrı geliştirme süreci gerektirir.
Sunucu tabanlı kurumsal sistemler	Proje kapsamının dışındadır.

Trade-Off'lar

Avantajlar:

- Kurulum gerektirmez.
- Platform bağımsızdır.
- Hızlı erişim sağlar.
- Güncelleme dağıtımı kolaydır.

Dezavantajlar:

- Tarayıcı verileri temizlenirse bilgiler kaybolabilir.
- Tarayıcı uyumluluğu testleri gerektirir.
- Sistem kaynaklarına erişim sınırlıdır.

Öğrenme Kaynakları

- [MDN Web Docs](#)
- [Can I Use](#)

Projedeki Rolü

Uygulama, tüm iş mantığı ve kullanıcı arayüzüyle birlikte doğrudan tarayıcı üzerinde çalışmaktadır. Kullanıcılar herhangi bir yazılım kurmadan projeyi açabilmekte, görevlerini yönetebilmekte, verilerini kaydedebilmekte ve dışa aktarabilmektedir. Tarayıcı, bu proje için hem çalışma ortamı hem de temel platform görevi üstlenmektedir.

4.7. Reddedilen Teknoloji Kararları

Yazılım geliştirme sürecinde kullanılan teknolojiler kadar değerlendirilen ancak bilinçli olarak tercih edilmeyen teknolojiler de önem taşımaktadır. Bu kararlar, projenin ihtiyaçları, geliştirme süresi, öğrenme hedefleri ve bakım kolaylığı dikkate alınarak verilmiştir. Bu bölümde, değerlendirilmiş ancak mevcut proje kapsamında kullanılmayan başlıca teknolojiler ve seçilmeme nedenleri sunulmaktadır.

Tablo 19: Proje İçin Farklı Alternatifler ve Seçilmeme Nedenleri

Teknoloji	Kategori	Seçilmeme Nedeni
React	Frontend Framework	Projenin temel amacı saf JavaScript ile uygulama geliştirmek olduğu için framework bağımlılığı oluşturmak istenmemiştir.
Vue.js	Frontend Framework	Küçük ölçekli proje için ek öğrenme yükü ve yapı karmaşıklığı oluşturmuştur.
Angular	Frontend Framework	Proje ölçeğine göre fazla kapsamlı ve ağır bir çözümdür.
Redux	State Management	Mevcut merkezi state yapısı proje ihtiyaçlarını yeterli düzeyde karşılamıştır.
SortableJS	Drag & Drop	HTML5 Drag & Drop API ile aynı işlev harici bağımlılık olmadan sağlanmıştır.
IndexedDB	Veri Saklama	localStorage, mevcut veri hacmi için daha basit ve yeterli bir çözüm sunmuştur.
Firebase	Backend / Cloud	Proje tamamen istemci taraflı çalışacak şekilde tasarlanmıştır.
Bootstrap	UI Framework	Özgün tasarım ve tam kontrol amacıyla saf CSS tercih edilmiştir.

Tail wind CSS	CSS Framework	Proje kapsamı için ek bağımlılık gerekli görülmemiştir.
Elec tron	Packagi ng	Kurulum gerektirmeyen tarayıcı tabanlı yapı yeterli görülmüştür.

Genel Değerlendirme

Reddedilen teknoloji kararları incelendiğinde, proje boyunca temel önceliğin sadelik, düşük bağımlılık, anlaşılabilirlik ve eğitim odaklı geliştirme yaklaşımı olduğu görülmektedir. Projenin amacı yalnızca çalışan bir uygulama ortaya koymak değil, aynı zamanda modern web teknolojilerinin çekirdek prensiplerini doğrudan uygulamaktır.

Bu nedenle, harici framework ve kütüphaneler yerine mümkün olduğunca yerleşik tarayıcı API'leri ve saf JavaScript tercih edilmiştir. Böylece hem uygulamanın çalışma mantığı daha şeffaf hale getirilmiş hem de kod tabanının bakım ve genişletilebilirliği artırılmıştır.

5. SİSTEM MİMARİSİ

Bu bölümde geliştirilen Vanilla JS Kanban uygulamasının genel mimari yapısı ve bileşenler arasındaki veri akışı açıklanmaktadır. Uygulama, tamamen istemci taraflı çalışan ve modüler bir mimari yaklaşımıyla geliştirilen bir web uygulamasıdır. Kullanıcı arayüzü, iş mantığı, veri saklama ve dışa aktarma işlemleri birbirinden ayrılmış bileşenler halinde tasarlanmıştır.

Sistem mimarisi; kullanıcı etkileşimlerinin nasıl işlendiğini, verilerin hangi yapılar içerisinde tutulduğunu ve bu verilerin tarayıcı üzerinde nasıl kalıcı hale getirildiğini göstermektedir. Ayrıca uygulama içerisinde kullanılan modüller arasındaki sorumluluk dağılımı ve veri akışı bu bölümde detaylandırılmaktadır.

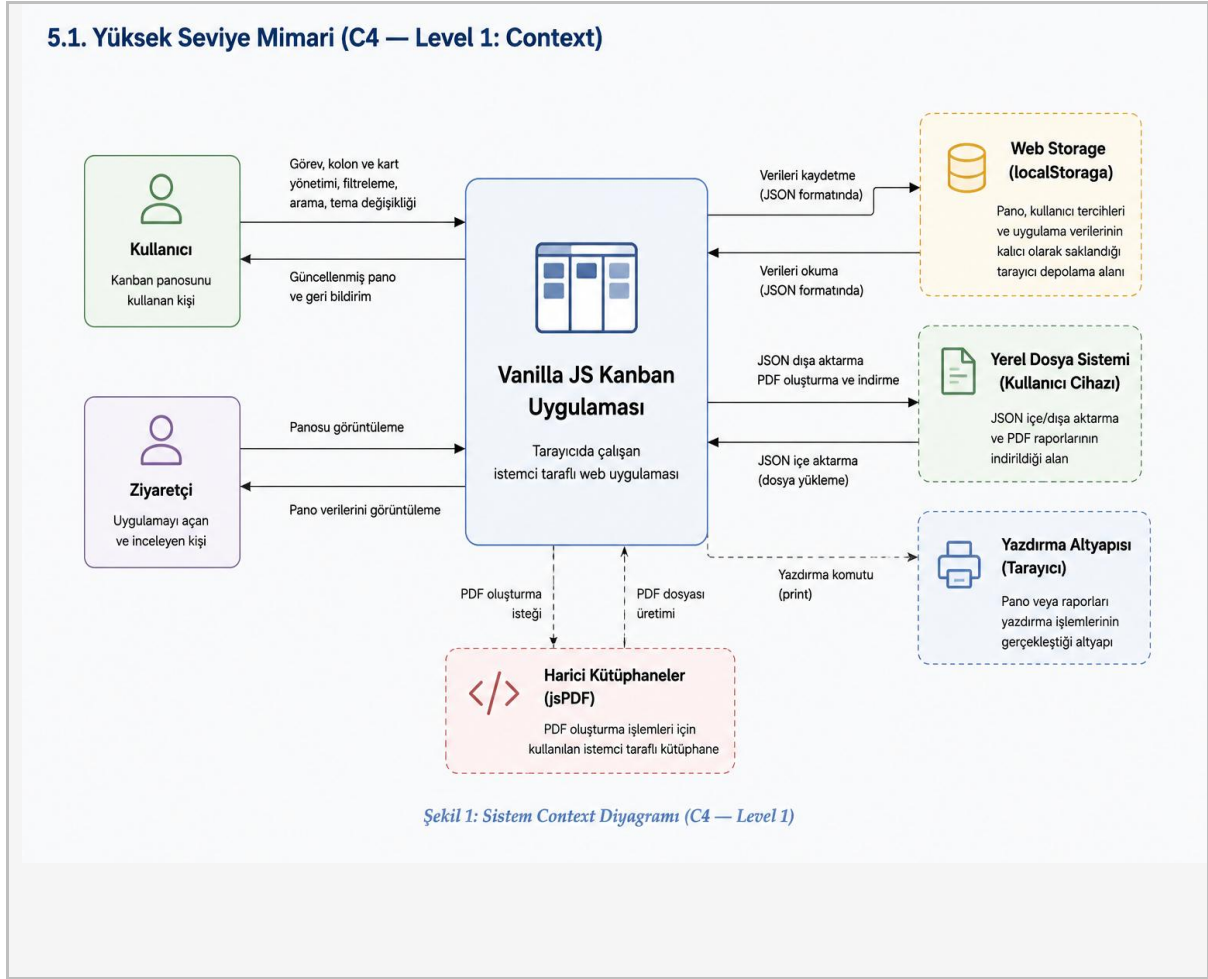
Bu mimari yaklaşım sayesinde uygulama:

- okunabilir,
- sürdürülebilir,
- test edilebilir,
- genişletilebilir

bir yapıya kavuşmuştur.

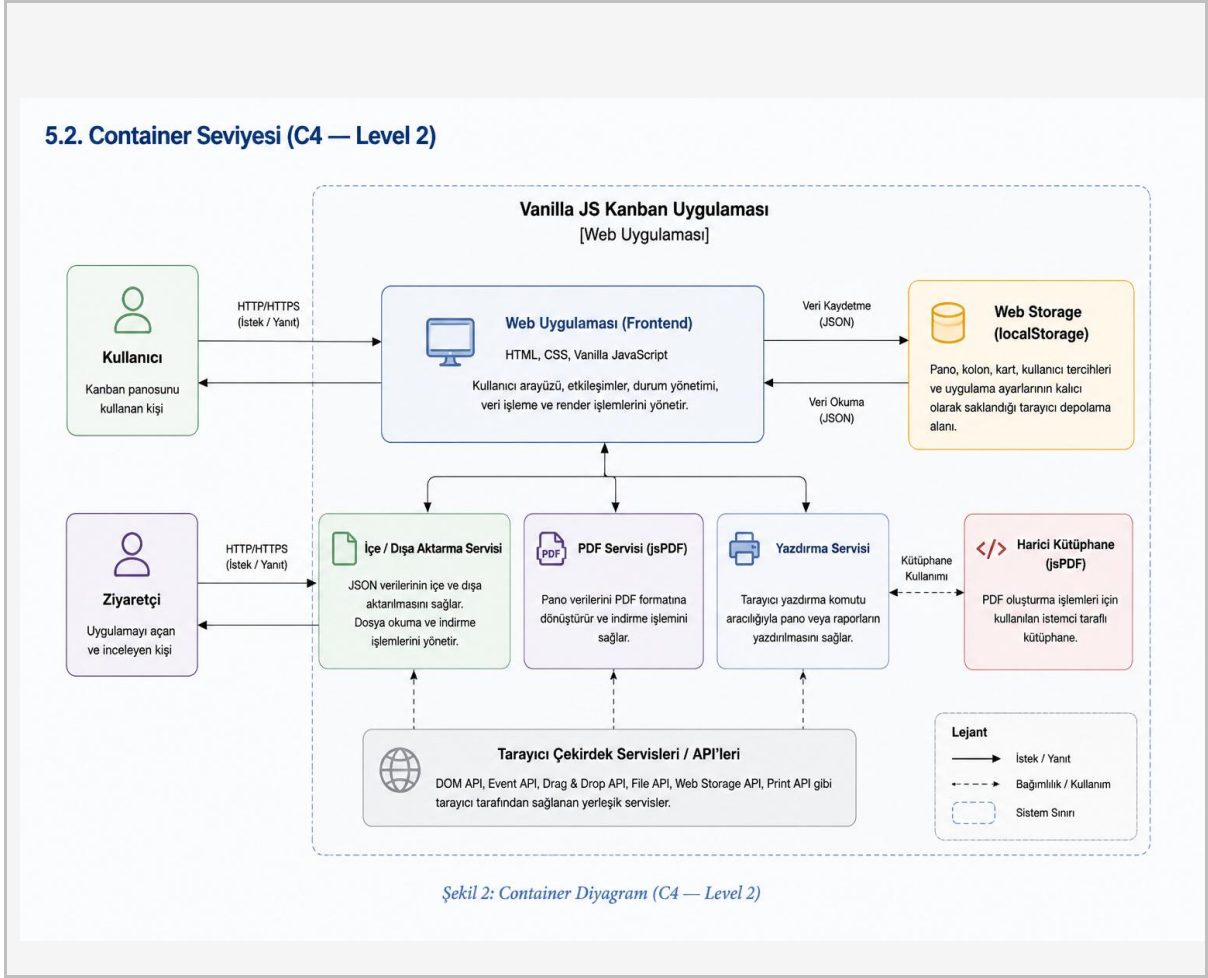
Bu bölümün alt başlıklarında sırasıyla yüksek seviye mimari, bileşenler arası ilişkiler, veri akış diyagramları, dağıtım topolojisi ve mimari karar kayıtları (ADR) sunulacaktır.

5.1. Yüksek Seviye Mimari (C4 — Level 1: Context)



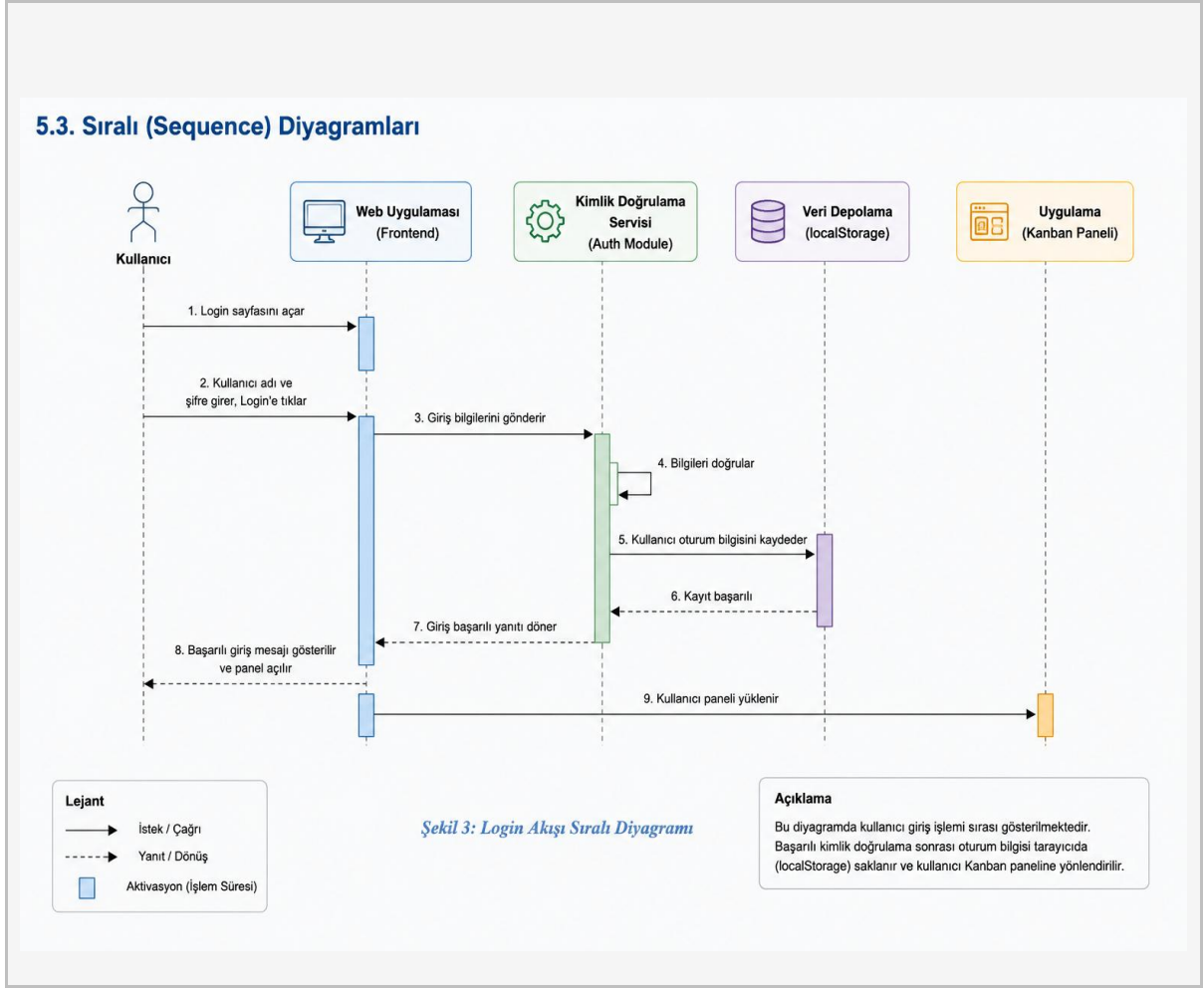
Şekil 1: Sistem Context Diyagramı (C4 — Level 1)

5.2. Container Seviyesi (C4 — Level 2)



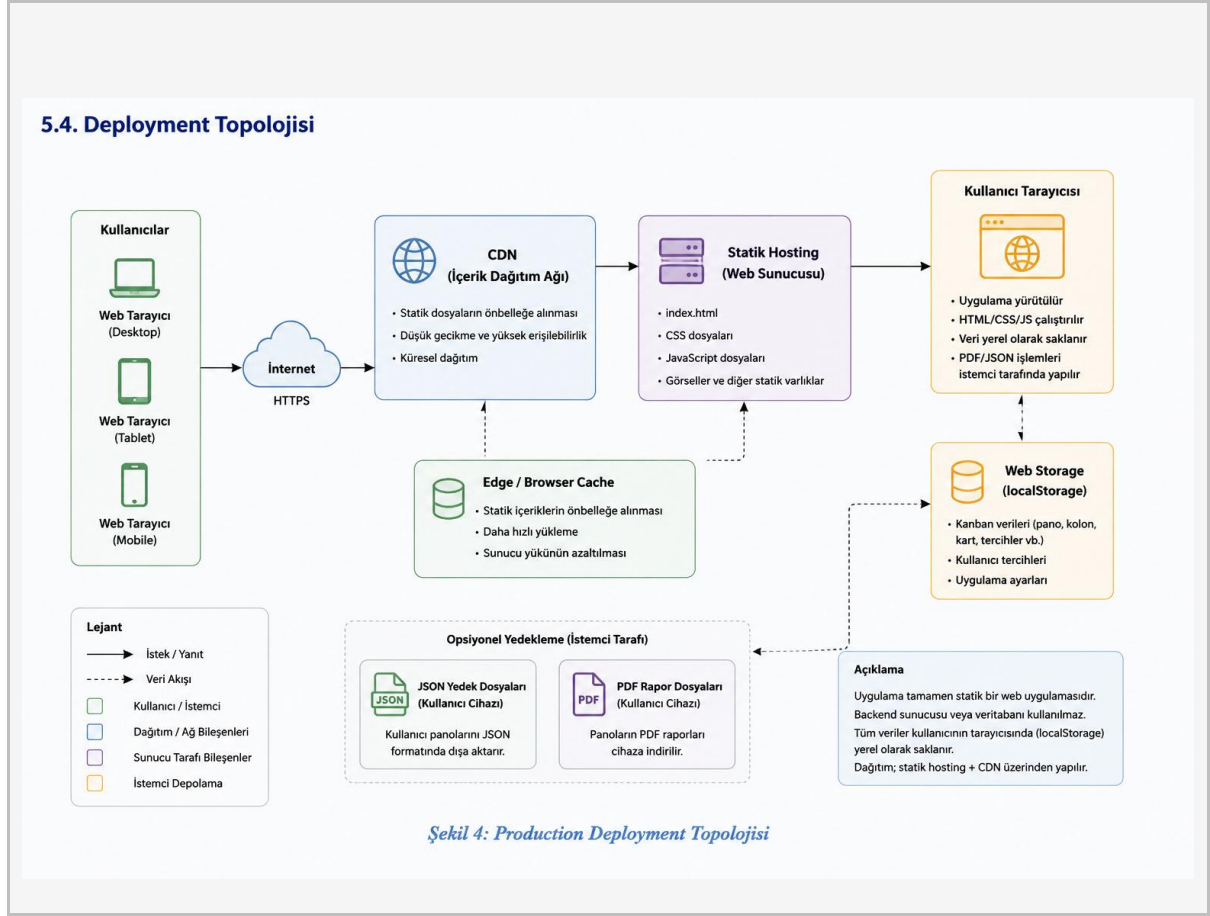
Şekil 2: Container Diyagramı (C4 — Level 2)

5.3. Sıralı (Sequence) Diyagramları



Şekil 3: Login Akışı Sıralı Diyagramı

5.4. Deployment Topolojisi



Şekil 4: Production Deployment Topolojisi

5.5. Mimari Karar Kayıtları (Architecture Decision Records — ADR)

Bu bölümde, proje geliştirme sürecinde alınan temel mimari kararlar ve bu kararların gerekçeleri özetlenmektedir. Mimari Karar Kayıtları (ADR), sistem tasarımını etkileyen önemli teknik tercihlerin belgelenmesini sağlayan kısa ve yapılandırılmış kayıtlardır. Bu yaklaşım sayesinde, hangi kararın alındığı, kararın mevcut durumu ve alınma tarihi sistematik olarak izlenebilmektedir.

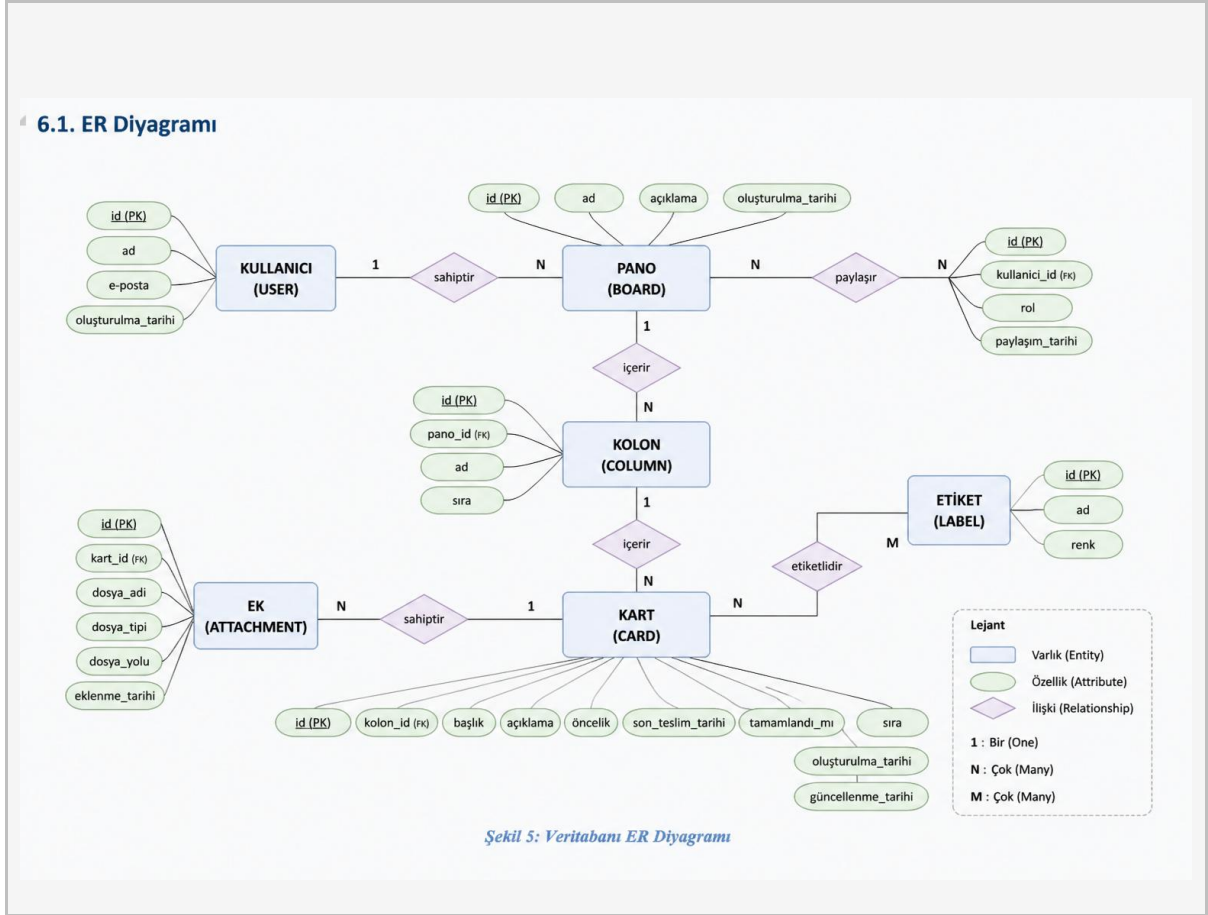
Vanilla JS Kanban uygulamasında alınan kararlar; projenin sade, istemci tarafı, düşük bağımlılığa sahip ve çevrimdışı çalışabilen bir yapıda geliştirilmesi hedefi doğrultusunda şekillendirilmiştir. Tablo 20’de projeyi doğrudan etkileyen başlıca mimari kararlar özetlenmiştir.

Tablo 20: ADR Özet Listesi

ADR No	Karar	Durum	Tarih
ADR-001	Frontend geliştirme yaklaşımı olarak Vanilla JavaScript kullanılması	Accepted	2026-04-29
ADR-002	Veri kalıcılığı için localStorage mekanizmasının kullanılması	Accepted	2026-04-30
ADR-003	Sürükle-bırak işlemleri için HTML5 Drag & Drop API'nin tercih edilmesi	Accepted	2026-05-02
ADR-004	PDF raporlama için jsPDF kütüphanesinin kullanılması	Accepted	2026-04-03
ADR-005	Veri yedekleme için JSON Import / Export mekanizmasının uygulanması	Accepted	2026-04-03

6. VERİ MODELİ VE API TASARIMI

6.1. ER Diyagramı



Şekil 5: Veritabanı ER Diyagramı

6.2. Tablo Tanımları

6.2 Tablo Tanımları

Bu bölümde, uygulamada kullanılan temel veri yapılarının tablo formatındaki tanımları sunulmaktadır. Proje geleneksel bir ilişkisel veritabanı kullanmamakta; veriler JavaScript nesneleri halinde tutulmakta ve JSON formatında `localStorage` üzerinde saklanmaktadır. Ancak veri modelinin daha anlaşılır olması amacıyla, bu yapılar ilişkisel veritabanı mantığına benzer şekilde tablo formatında doküman edilmiştir.

Bu yaklaşım sayesinde her veri yapısındaki alanların:

- veri tipi,
- boş değer alıp alamadığı,
- varsayılan değerleri,
- anahtar bilgileri,

- açıklamaları

sistematiik olarak gösterilebilmektedir.

Projede en temel veri yapıları aşığıdaki tablolar üzerinden temsil edilmektedir:

1. users
2. boards
3. columns
4. cards
5. checklist_items

Aşığıda her tablo için detaylı alan tanımları sunulmaktadır.

Tablo 21: Örnek Tablo: users

Kolon	Tip	Null?	Default	Index	Açıklama
id	String	✗	generateId())	PK	Benzersiz Kullanıcı Kimliği
name	String	✗	—	—	Kullanıcı Adı
avatar	String	✓	""	—	Profil Görsel Yolu
color	String	✓	"#4F46E5"	—	Kullanıcı Renk Kodu
created_at	DateTime	✗	new Date()	—	Oluşturulma Tarihi

Tablo 22: Örnek Tablo: boards

Kolon	Tip	Null?	Default	Index	Açıklama
id	String	✗	generateId())	PK	Pano Kimliği
title	String	✗	—	—	Pano Başlığı
created_at	DateTime	✗	new Date()	—	Oluşturulma Tarihi

Tablo 23: Örnek Tablo: boards

Kolon	Tip	Null?	Default	Index	Açıklama
id	String	X	generateId()	PK	Kolon kimliği
board_id	String	X	—	FK	Bağlı olduğu pano
title	String	X	—	—	Kolon adı
order	Number	X	0	—	Görüntüleme sırası
created_at	DateTime	X	new Date()	—	Oluşturulma tarihi

Tablo 24: Örnek Tablo: cards

Kolon	Tip	Null?	Default	Index	Açıklama
id	String	X	generateId()	PK	Kart kimliği
column_id	String	X	—	FK	Bağlı olduğu kolon
title	String	X	—	—	Kart başlığı
description	Text	✓	""	—	Kart açıklaması
due_date	Date	✓	null	—	Son teslim tarihi
priority	String	✓	"medium"	—	Öncelik seviyesi
assigned_user_id	String	✓	null	FK	Atanan kullanıcı
order	Number	X	0	—	Kart sırası
created_at	DateTime	X	new Date()	—	Oluşturulma tarihi
updated_at	DateTime	X	new Date()	—	Son güncelleme tarihi

Tablo 25: Örnek Tablo: checklist_items

Kolon	Tip	Null?	Default	Index	Açıklama
id	String	X	generateId()	PK	Checklist öge kimliği
card_id	String	X	—	FK	Bağlı olduğu kart
text	String	X	—	—	Görev açıklaması
completed	Boolean	X	false	—	Tamamlanma durumu
order	Number	X	0	—	Liste sırası

6.3. İndeks Stratejisi

Projede veriler ilişkisel bir veritabanında değil, JavaScript nesneleri olarak `localStorage` üzerinde tutulmaktadır. Bu nedenle fiziksel anlamda SQL indeksleri oluşturulmamaktadır. Ancak veri erişim performansını artırmak amacıyla, ilişkisel veritabanı mantığına benzer şekilde benzersiz kimlik alanları ve referans alanları kullanılmaktadır.

Bu bölümde, veri modelinde mantıksal olarak indekslenmesi gereken alanlar ve bu alanların kullanım amaçları özetlenmiştir. Özellikle benzersiz kimlikler, kullanıcı eşleştirmeleri ve ilişkisel bağlantılar uygulama içerisinde hızlı veri erişimi sağlamaktadır.

Tablo 26: İndeks Kararları

Tablo	İndeks	Amaç
users	id (primary key)	Kullanıcı kayıtlarına hızlı erişim
boards	id (primary key)	Panolara benzersiz erişim
columns	id (primary key)	Kolon kayıtlarına hızlı erişim
columns	board_id	Belirli bir panoya ait kolonları listeleme
cards	id (primary key)	Kart kayıtlarına hızlı erişim
cards	column_id	Belirli bir kolondaki kartları listeleme
cards	assigned_user_id	Kullanıcıya atanmış kartları bulma
checklist_items	id (primary key)	Checklist öğelerine erişim
checklist_items	card_id	Belirli bir karta ait checklist öğelerini ekleme

Bu mantıksal indeksleme yaklaşımı, uygulamanın veri ilişkilerini açık bir şekilde tanımlamakta ve ileride gerçek bir veritabanına geçiş yapılması durumunda uygun indeks tasarımına temel oluşturmaktadır. Ayrıca veri erişim örüntülerinin önceden düşünülmesini sağlayarak sistem tasarımının daha sağlam bir şekilde kurgulanmasına katkıda bulunmaktadır.

6.4. REST API Endpoint Listesi

[Minimum 10 endpoint dokümanate ediniz. OpenAPI 3.1 formatındaki spec'i repo'nuzdaki bir `openapi.yaml` dosyasına da ekleyiniz.]

Tablo 27: Ana API Endpoint'leri

Metot	URL	Açıklama	Auth
POST	/api/auth/register	Yeni kullanıcı kaydı oluşturur	Public
POST	/api/auth/login	Kullanıcı girişi yapar ve token üretir	Public

POST	/api/auth/refresh	Access token yenileme	Refresh Token
GET	/api/me	Oturum açmış kullanıcı bilgilerini döner	JWT
GET	/api/boards	Kullanıcının panolarını listeler	JWT
POST	/api/boards	Yeni pano oluşturur	JWT
GET	/api/boards/:id	Belirli bir panonun detayını getirir	JWT
PATCH	/api/boards/:id	Pano bilgilerini günceller	JWT
DELETE	/api/boards/:id	Panoyu siler	JWT
GET	/api/boards/:id/export	Panoyu JSON olarak dışa aktarır	JWT
GET	/api/boards/:id/pdf	Panoyu PDF raporu olarak oluşturur	JWT
POST	/api/boards/import	JSON dosyasından pano içe aktarır	JWT
GET	/api/users	Kullanıcı listesini getirir	JWT
POST	/api/cards/:id/checklist	Karta checklist öğesi ekler	JWT
PATCH	/api/cards/:id	Kartı farklı kolona taşır	JWT

6.5 Authentication ve Authorization

Bu proje mevcut haliyle tamamen istemci taraflı çalışan bir web uygulaması olduğundan, sunucu tabanlı bir kimlik doğrulama (authentication) ve yetkilendirme (authorization) mekanizması kullanılmamaktadır. Uygulama, tek kullanıcı odaklı olarak tasarlanmış olup tüm veriler kullanıcının tarayıcısındaki `localStorage` üzerinde saklanmaktadır.

Bu nedenle kullanıcıların sisteme giriş yapması, oturum açması veya rol bazlı erişim kontrolü uygulanması gerekmemektedir. Uygulamayı açan kullanıcı doğrudan Kanban panosuna erişebilmekte ve kendi verileri üzerinde işlem yapabilmektedir.

Bununla birlikte, sistemin gelecekte çok kullanıcılı bir yapıya dönüştürülmesi durumunda aşağıdaki kimlik doğrulama ve yetkilendirme mekanizmalarının uygulanması planlanabilir:

Authentication (Kimlik Doğrulama)

- Kullanıcı kayıt ve giriş sistemi
- E-posta ve parola ile oturum açma
- Parolaların güvenli biçimde hashlenmesi (örneğin `bcrypt`)

- JWT tabanlı access ve refresh token yapısı
- Oturum yenileme ve çıkış mekanizması

Authorization (Yetkilendirme)

- Kullanıcıların yalnızca kendi panolarına erişebilmesi
- Rol bazlı yetkilendirme (owner, editor, viewer)
- Kaynak bazlı erişim kontrolleri
- API seviyesinde token doğrulama

Güvenlik Önlemleri

- HTTPS üzerinden güvenli iletişim
- Input validation ve sanitization
- Rate limiting
- CSRF ve XSS korumaları
- Güvenli token saklama yöntemleri

Mevcut sürümde uygulama yerel olarak çalıştığı için kullanıcı verileri üçüncü taraf sunuculara gönderilmemekte ve tüm işlemler istemci tarafında gerçekleşmektedir. Bu yaklaşım, kişisel kullanım senaryolarında veri gizliliği açısından önemli bir avantaj sağlamaktadır.

Sonuç olarak, mevcut projede aktif bir authentication/authorization altyapısı bulunmamakla birlikte, sistem mimarisi gelecekte kullanıcı hesapları, rol yönetimi ve güvenli API erişimi eklenecek şekilde genişletilebilecek biçimde tasarlanmıştır.

6.6. Rate Limiting Politikası

Bu proje mevcut sürümünde istemci taraflı çalışan bir web uygulaması olduğundan, gerçek anlamda sunucu tabanlı bir rate limiting mekanizması uygulanmamaktadır. Çünkü uygulamada backend API bulunmamakta ve tüm işlemler kullanıcı tarayıcısı üzerinde gerçekleştirilmektedir.

Bununla birlikte, sistemin gelecekte çok kullanıcı ve REST API destekli bir yapıya dönüştürülmesi durumunda, kötüye kullanımı önlemek, sunucu kaynaklarını korumak ve güvenliği artırmak amacıyla rate limiting politikalarının uygulanması önerilmektedir.

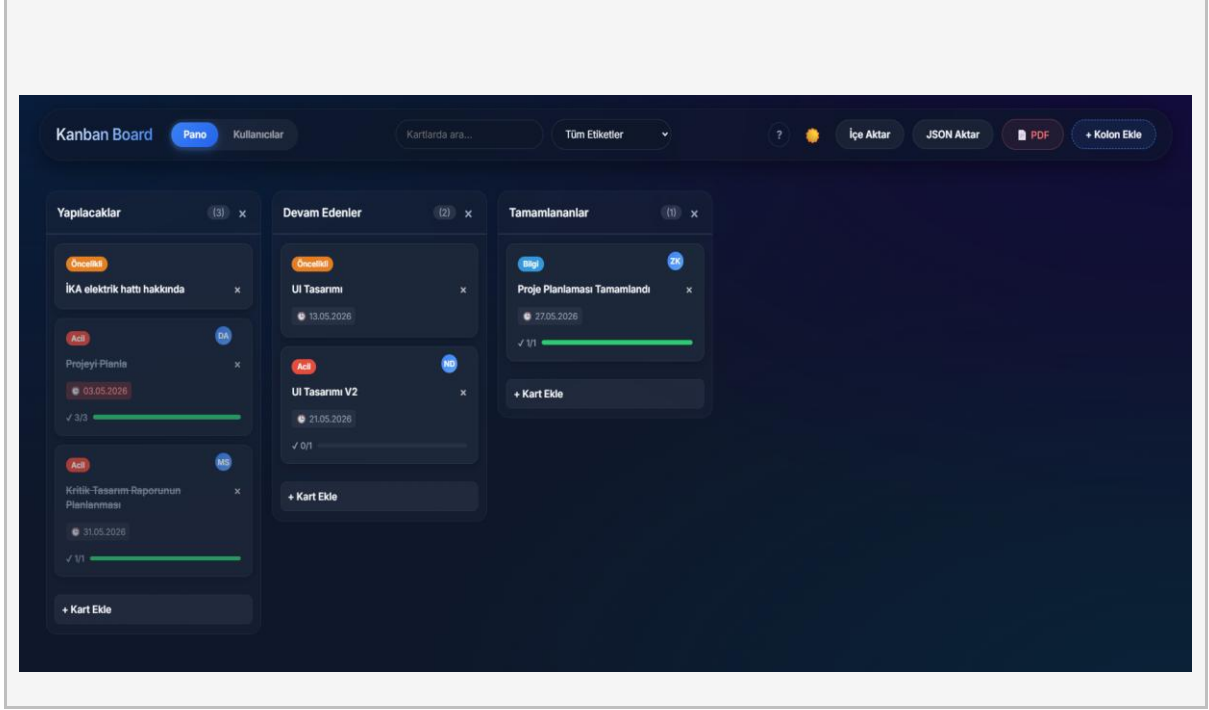
Tablo 28: Rate Limit Kuralları

Endpoint grubu	Limit
Auth (login/register)	5/dk/IP

API okuma (GET)	100/dk/kullanıcı
API yazma (POST, PATCH, DELETE)	30/dk/kullanıcı
Token yenileme (/refresh)	10/dk/kullanıcı
Dosya içe aktarma	5/dk/kullanıcı
PDF oluşturma	10/dk/kullanıcı

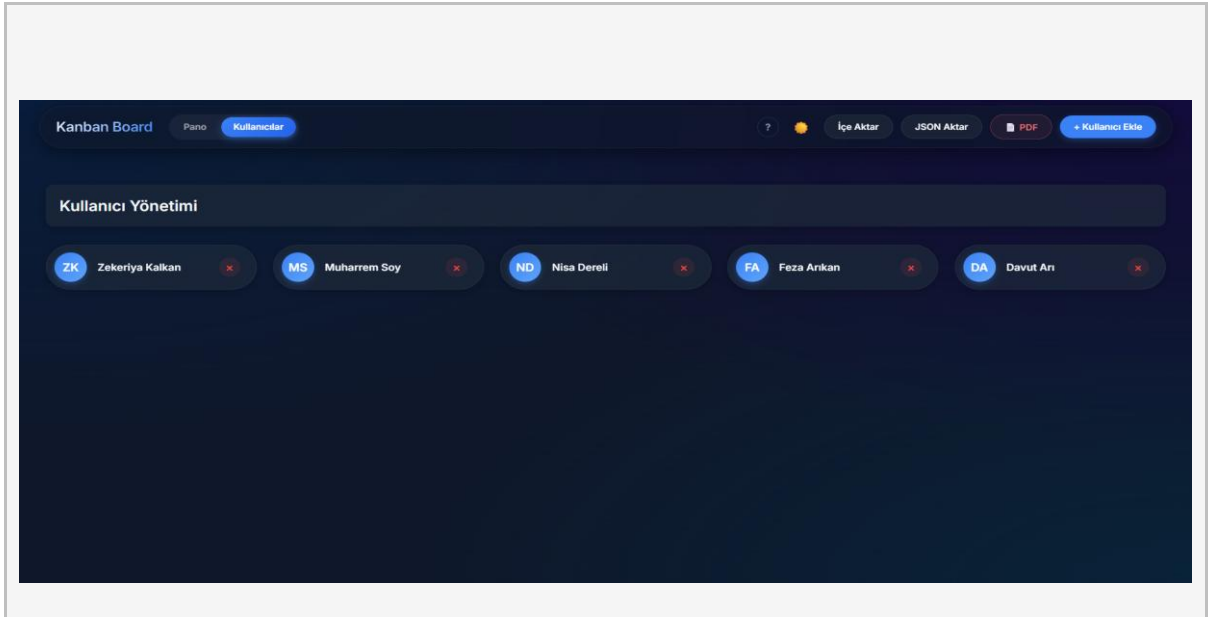
7. KULLANICI ARAYÜZÜ TASARIMI

7.1. Bilgi Mimarisi (Sitemap)



Şekil 6: Uygulama Site Haritası

7.2. Kullanıcı Akışı (User Flow)



Şekil 7: Ana Kullanıcı Akışı (User Flow)

7.3. Design System

7.3 Design System

Bu bölümde uygulamanın görsel tasarımında kullanılan temel renkler ve tipografi kuralları sunulmaktadır. Tasarım sistemi; arayüzde görsel tutarlılığın sağlanması, bileşenlerin standartlaştırılması ve kullanıcı deneyiminin iyileştirilmesi amacıyla oluşturulmuştur.

Tablo 29: Renk Paleti

Role	Adı	HEX	Kullanım
Primary 500	Indigo	#4F46E5	Birincil butonlar ve ana aksiyonlar
Primary 600	Indigo Dark	#4338CA	Hover ve aktif durumlar
Gray 50	Background	#F9FAFB	Sayfa arka planı
Gray 100	Surface	#F9FAFB	Kart ve panel arka planları
Gray 900	Text primary	#111827	Ana metin rengi
Gray 500	Text Secondary	#6B7280	Yardımcı metinler
Success	Emerald	#10B981	Başarı toast
Warning	Amber	#F59E0B	Uyarı durumları ve yaklaşan teslim tarihleri
Danger	Red	#EF4444	Hata / silme
Info	Sky Blue	#0EA5E9	Bilgilendirme mesajları

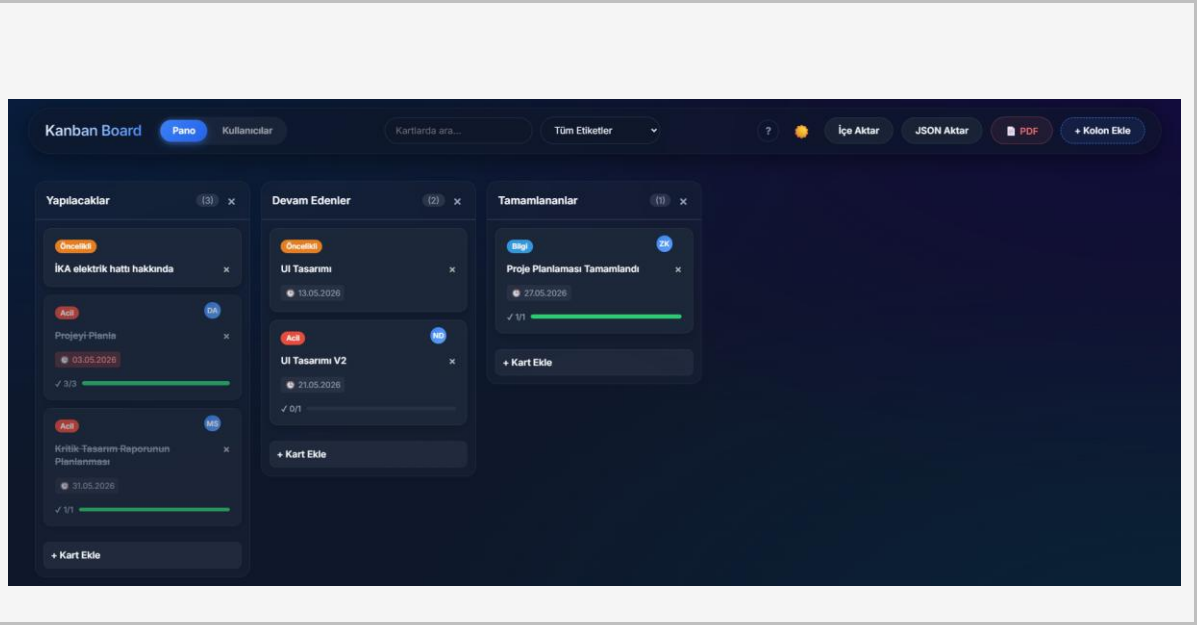
Tablo 30: Tipografi Skalası

Seviye	Boyut (px)	Line-height	Ağırlık	Kullanım
Display	48	1.1	700	Hero başlığı
H1	36	1.2	700	Sayfa başlığı
H2	30	1.3	600	Bölüm başlığı
H3	24	1.4	600	Alt başlık
H4	20	1.4	600	Kart veya model başlıkları
Body	16	1.5	400	Paragraf
Small	14	1.4	400	Etiketler, rozetler ve dipnotlar

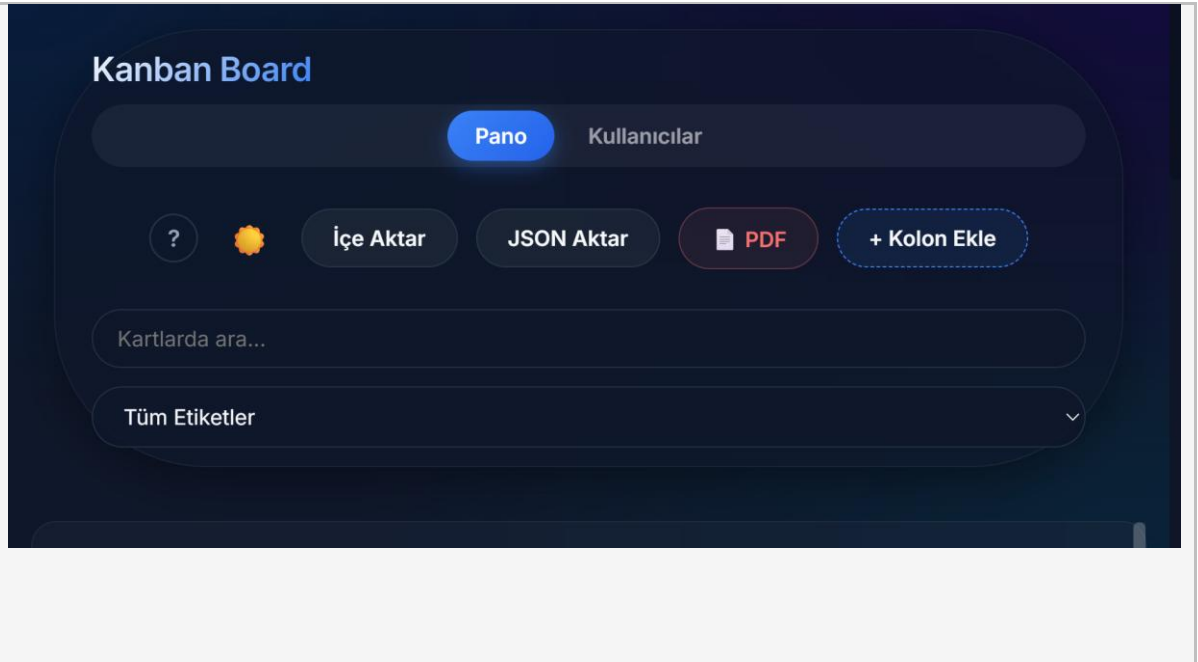
Caption	12	1.4	500	Etiket, dipnot
----------------	----	-----	-----	----------------

Uygulamada renkler CSS değişkenleri (CSS Variables) aracılığıyla merkezi olarak yönetilmiştir. Bu sayede dark/light tema desteği kolaylıkla uygulanmış ve tüm bileşenlerde görsel tutarlılık sağlanmıştır. Tipografi sistemi ise okunabilirliği artırmak ve farklı içerik seviyeleri arasında net bir hiyerarşi oluşturmak amacıyla tasarlanmıştır.

7.4. Wireframe'ler ve Mockuplar



Şekil 8: Landing Sayfası Wireframe



Şekil 9: Dashboard Mockup

7.5. Responsive Tasarım Stratejisi

Uygulama, farklı ekran boyutlarında tutarlı ve kullanılabilir bir deneyim sunmak amacıyla responsive tasarım prensiplerine uygun olarak geliştirilmiştir. CSS Grid, Flexbox ve media query yapıları kullanılarak kullanıcı arayüzü; mobil cihazlardan geniş masaüstü ekranlara kadar ölçeklenebilir şekilde tasarlanmıştır.

Responsive tasarım yaklaşımında temel hedefler şunlardır:

- İçeriğin ekran boyutuna göre yeniden düzenlenmesi,
- Dokunmatik cihazlarda rahat kullanılabilir arayüz elemanları sunulması,
- Küçük ekranlarda gereksiz görsel karmaşıklığın azaltılması,
- Büyük ekranlarda mevcut alanın verimli şekilde değerlendirilmesi.

Tablo 31: Breakpoint'ler

Cihaz	Min Genişlik	Notlar
Mobile	375 px	Tek kolon, hamburger nav, dokunmatik 44×44 px
Tablet	768 px	İki kolon, sidebar açılır
Laptop	1024 px	Kompakt layout
Desktop	1280 px	Tam genişlik tasarımı

Responsive Tasarım Kararları

- **Kanban kolonları**, dar ekranlarda yatay kaydırılabilir yapıdadır.
- **Modal pencereler**, mobil cihazlarda ekranın büyük bölümünü kaplayacak şekilde ölçeklenmektedir.
- **Navigasyon bileşenleri**, küçük ekranlarda sadeleştirilmiş menü yapısına dönüşmektedir.
- **Buton ve form elemanları**, dokunmatik kullanım için yeterli boyutta tasarlanmıştır.
- **Tipografi**, ekran boyutuna göre okunabilirliği koruyacak şekilde optimize edilmiştir.

Bu yaklaşım sayesinde uygulama, masaüstü bilgisayarlar, tabletler ve mobil cihazlarda ek geliştirme gerektirmeden kullanılabilir. Responsive tasarım stratejisi, kullanıcı deneyimini artırırken uygulamanın erişilebilirliğini ve platform bağımsızlığını da güçlendirmektedir.

7.6. Erişilebilirlik (a11y)

7.6 Erişilebilirlik (a11y)

Uygulama geliştirme sürecinde, farklı kullanıcıların sistemi rahatlıkla kullanabilmesi amacıyla erişilebilirlik (accessibility) prensipleri dikkate alınmıştır. Tasarım ve geliştirme kararları, [WCAG 2.1 AA standartları](#) temel alınarak değerlendirilmiştir. Amaç; klavye ile tam kullanım, yeterli renk kontrastı ve yardımcı teknolojilerle uyumlu bir kullanıcı deneyimi sunmaktır.

Uygulanan Erişilebilirlik Önlemleri

Renk Kontrastı

- Metin ve arka plan renkleri, WCAG 2.1 AA standardında önerilen minimum 4.5:1 kontrast oranını sağlayacak şekilde seçilmiştir.
- Hata, başarı ve uyarı mesajlarında yalnızca renk değil, metinsel açıklamalar da kullanılmıştır.

Klavye Navigasyonu

- Tüm butonlar, form elemanları ve modal pencereler klavye ile erişilebilir durumdadır.
- Tab ve Shift + Tab ile odak geçişi yapılabilir.
- Enter ve Space tuşları ile temel işlemler tetiklenebilir.
- Esc tuşu ile modal pencereler kapatılabilir.

ARIA Kullanımı

- Modal bileşenlerinde role="dialog" ve aria-modal="true" nitelikleri kullanılmıştır.
- Form alanları açıklayıcı label etiketleri ile ilişkilendirilmiştir.
- İkon tabanlı butonlara aria-label tanımlanmıştır.

Alternatif Metinler

- Anlam taşıyan görseller ve kullanıcı avaturları için uygun alt metinleri sağlanmıştır.

Hareket Hassasiyeti

- Kullanıcıların sistem tercihlerine saygı göstermek amacıyla prefers-reduced-motion medya sorgusu dikkate alınmıştır.

- Kritik animasyonlar gerektiğinde azaltılabilir veya devre dışı bırakılabilir.

Responsive ve Dokunmatik Kullanım

- Dokunmatik hedefler minimum 44×44 px olacak şekilde tasarlanmıştır.
- Küçük ekranlarda okunabilirlik ve kullanılabilirlik korunmuştur.

Test ve Doğrulama

Erişilebilirlik değerlendirmeleri aşağıdaki araçlar kullanılarak yapılmıştır:

- [Google Lighthouse](#)
- [axe DevTools](#)
- Manuel klavye navigasyonu testleri

Bu araçlar sayesinde kontrast, ARIA kullanımı, odak yönetimi ve semantik HTML yapısı doğrulanmıştır.

Genel Değerlendirme

Uygulama, temel erişilebilirlik gereksinimlerini karşılayacak şekilde tasarlanmıştır. Özellikle klavye navigasyonu, yeterli renk kontrastı ve semantik HTML kullanımı sayesinde sistemin farklı kullanıcı profilleri tarafından daha rahat kullanılabilmesi sağlanmıştır. Gelecekte ekran okuyucu testlerinin genişletilmesi ve otomatik erişilebilirlik denetimlerinin CI/CD sürecine eklenmesi planlanabilir.

8. GÜVENLİK, PERFORMANS VE TEST

8.1. Güvenlik (OWASP Top 10)

Tablo 31: OWASP Top 10 (2021) Uygulama Matrisi

Kod	Risk	Bu projedeki önlemim
A01	Broken Access Control	Gelecekte JWT tabanlı authentication ve rol bazlı yetkilendirme (RBAC) uygulanacaktır.
A02	Cryptographic Failures	HTTPS kullanımı, güvenli token saklama ve hassas verilerin şifrenmesi planlanmaktadır.
A03	Injection (SQL / XSS)	Kullanıcı girdileri doğrulanmakta, DOM'a eklenen içerikler güvenli şekilde işlenmektedir.
A04	Insecure Design	Mimari kararlar ADR, threat modeling ve güvenlik odaklı tasarım yaklaşımıyla belgelenmiştir.
A05	Security Misconfiguration	Varsayılan ayarlar gözden geçirilmiş, istemci tarafında gereksiz bağımlılıklardan kaçınılmıştır.
A06	Vulnerable Components	Kullanılan harici kütüphaneler (örneğin jsPDF) güncel sürümleriyle kullanılmaktadır.
A07	Identification and Authentication Failures	Gelecekte güçlü parola politikaları, rate limiting ve hesap kilitleme mekanizmaları uygulanabilir.
A08	Software and Data Integrity Failures	JSON import işlemlerinde veri yapısı doğrulanmakta, dosya formatı kontrol edilmektedir.
A09	Security Logging and Monitoring Failures	Gelecekte backend mimarisinde yapılandırılmış loglama ve izleme mekanizmaları eklenebilir.
A10	SSRF	Uygulama sunucu tarafı dış istek gerçekleştirmediği için mevcut sürümde bu risk bulunmamaktadır.

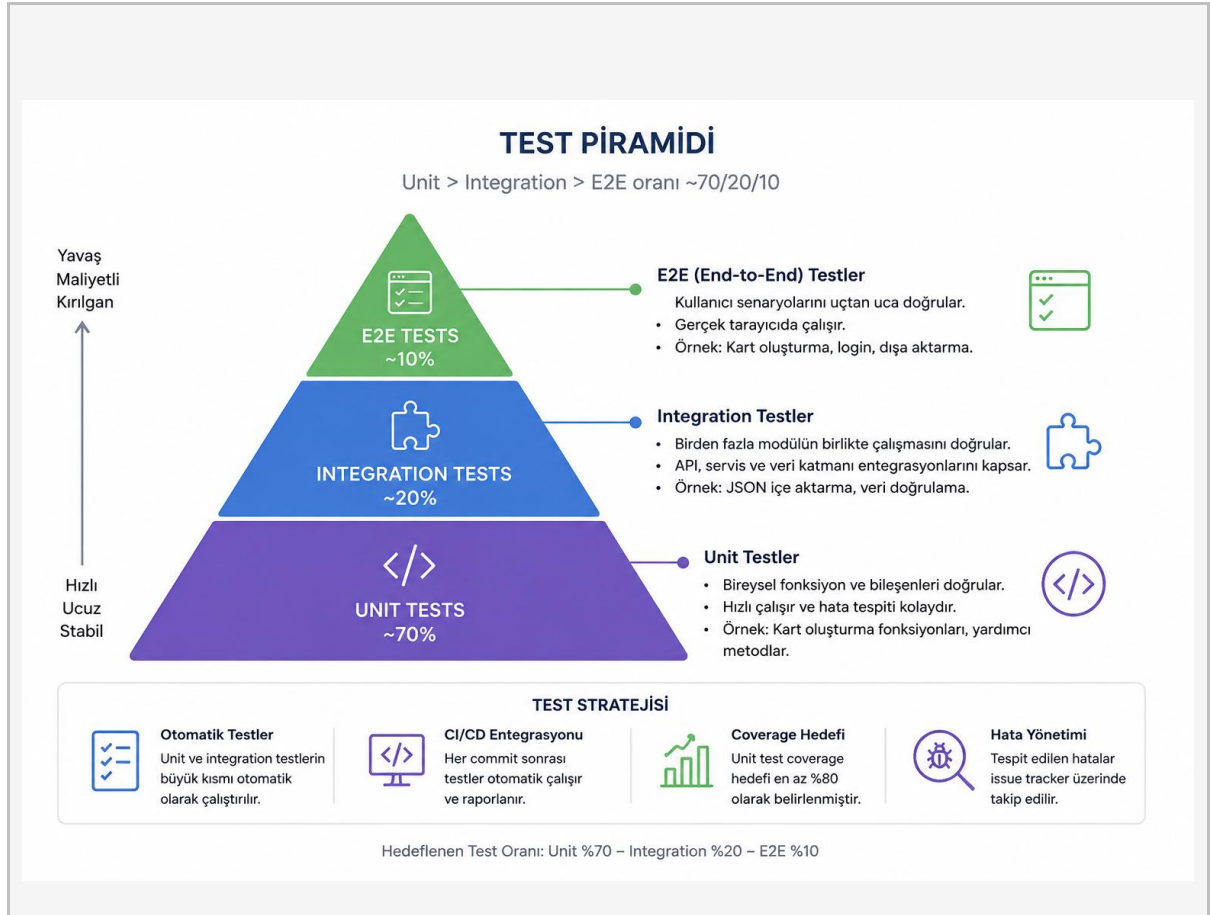
8.2. Performans Hedefleri

Tablo 32: Performans Metrikleri ve Hedefler

Metrik	Hedef	Ölçüm aracı
LCP (Largest Contentful Paint)	< 2.5 s	Lighthouse, Web Vitals

INP (Interaction to Next Paint)	< 200 ms	Web Vitals
CLS (Cumulative Layout Shift)	< 0.1	Web Vitals
Uygulama Başlatma Süresi	< 1 s	Chrome DevTools
Kart Taşıma Tepki Süresi	< 100 ms	Manuel Test
JSON Import Süresi	< 2 s	Manuel Test
PDF Oluşturma Süresi	< 3 s	Manuel Test
localStorage Kayıt Süresi	< 50 ms	Chrome DevTools
Bundle size (gzip)	< 200 KB	Kaynak dosya analizi

8.3. Test Stratejisi



Şekil 10: Test Piramidi

9. MALİYET, GELİR MODELİ VE GTM

9.1. Business Model Canvas

Tablo 33: Business Model Canvas (Alex Osterwalder)

Blok	İçerik
Customer Segments	Freelancer'lar, küçük ekipler, öğrenciler ve bireysel proje yöneticileri
Value Propositions	(1) Kurulum gerektirmeyen tarayıcı tabanlı Kanban uygulaması, (2) Verilerin cihaz üzerinde saklanması sayesinde gizlilik, (3) JSON/PDF export ile kolay yedekleme ve raporlama
Channels	GitHub, kişisel web sitesi, sosyal medya paylaşımları, ürün tanıtım videoları
Customer Relationships	Self-service kullanım, kullanım kılavuzu, GitHub üzerinden geri bildirim ve issue takibi
Revenue Streams	Ücretsiz temel sürüm, premium özellikler, kurumsal lisanslama, özel özelleştirme hizmetleri
Key Resources	Kaynak kod, tasarım sistemi, teknik dokümantasyon, GitHub deposu
Key Activities	Uygulama geliştirme, hata düzeltme, yeni özellik ekleme, kullanıcı geri bildirimlerinin değerlendirilmesi
Key Partnerships	GitHub, açık kaynak kütüphaneler (jsPDF), web hosting sağlayıcıları
Cost Structure	Geliştirme zamanı, bakım maliyeti, hosting giderleri, alan adı ve tanıtım maliyetleri

9.2. Gelir Modeli

9.2 Gelir Modeli

Bu proje için **Freemium (Ücretsiz + Premium)** gelir modeli önerilmektedir. Freemium modelinde temel özellikler ücretsiz olarak sunulurken, daha gelişmiş fonksiyonlar belirli bir abonelik ücreti karşılığında kullanıcılara sunulur. Bu yaklaşım, kullanıcıların ürünü risksiz şekilde denemesine olanak tanırken, ihtiyaçları arttıkça ücretli planlara geçiş yapmalarını teşvik eder.

Kanban uygulaması için temel görev yönetimi, kolon oluşturma, kart ekleme ve localStorage tabanlı veri saklama özellikleri ücretsiz planda sunulabilir. Gelişmiş raporlama, sınırsız pano, ekip paylaşımı, bulut senkronizasyonu ve gelişmiş yetkilendirme gibi özellikler ise ücretli planlara dahil edilebilir.

Bu modelin tercih edilmesinin başlıca nedenleri şunlardır:

- Kullanıcı edinme bariyerini düşürmesi,
- Ürünün hızlı şekilde yayılmasını sağlaması,
- Temel kullanıcıların ücretsiz kalabilmesi,
- İleri düzey ihtiyaçlar için sürdürülebilir gelir oluşturmaları.

Tablo 34: Fiyat Tabloması (3 tier)

Plan	Aylık Fiyat	Yıllık Fiyat	İçerik
Free	₺0	₺0	Temel Kanban özellikleri, 3 pano, JSON export/import
Pro	₺99	₺990	Sınırsız pano, PDF raporlama, gelişmiş filtreleme, dark mode
Business	₺299	₺2.990	Ekip yönetimi, rol bazlı yetkilendirme, bulut senkronizasyonu, öncelikli destek

Bu fiyatlandırma yapısı, bireysel kullanıcılar için ücretsiz erişim sunarken profesyonel kullanıcılar ve ekipler için ek değer sağlayan özellikler üzerinden gelir elde etmeyi hedeflemektedir. Freemium yaklaşımı, ürünün ölçeklenebilir ve ticari olarak sürdürülebilir bir modele dönüştürülmesine olanak tanımaktadır.

9.3. Maliyet Analizi

Tablo 35: 1. Yıl Toplam Sahip Olma Maliyeti (TCO)

Kalem	Tutar (₺)
Geliştirme (freelance eşdeğer)	120.000
12 ay altyapı (ortalama)	6.000
Domain + SSL	500

Pazarlama (ilk 3 ay kampanya)	15.000
Hukuk / danışmanlık	5.000
TOPLAM (1. yıl)	146.500

9.4. Unit Economics

Tablo 36: Temel Unit Ekonomi Metrikleri (Tahmin)

Metrik	Değer	Hesap
ARPU	₺99/ay	Aylık ortalama gelir (Pro kullanıcı)
Gross Margin	%85	(Revenue - COGS) / Revenue
Aylık Churn	%3	Aylık iptal oranı
LTV	₺2.805	$ARPU \times Gross\ Margin / Churn$
CAC	₺300	Google Ads + içerik pazarlama tahmini
LTV / CAC	9,35	≥ 3 sağlıklı
Payback period	3,6 ay	$CAC / (ARPU \times Gross\ Margin)$

9.5. Go-to-Market (GTM) Stratejisi

Vanilla JS Kanban uygulamasının pazara sunulmasında temel hedef, proje yönetimini sade, hızlı ve kurulum gerektirmeyen bir çözüm arayan bireysel kullanıcılar ile küçük ekiplerdir. Ürün, tarayıcı üzerinden doğrudan çalışması, verileri kullanıcının cihazında saklaması ve ücretsiz temel özellikler sunması sayesinde kullanıcı edinme açısından düşük giriş bariyerine sahiptir.

Hedef Kitle

İlk aşamada odaklanılacak kullanıcı segmentleri şunlardır:

- Freelancer'lar
- Yazılım geliştiriciler
- Üniversite öğrencileri
- Küçük startup ekipleri
- Kişisel görev takibi yapmak isteyen bireyler

İlk 100 Kullanıcıya Ulaşma Planı

Ürünün ilk kullanıcı kitlesine ulaşmak için aşağıdaki kanallar kullanılacaktır:

1. **GitHub Yayını:** Projenin kaynak kodu, ekran görüntüleri ve detaylı dokümantasyon ile birlikte GitHub üzerinde yayınlanacaktır.
2. **LinkedIn ve X (Twitter):** Projenin geliştirme süreci ve öne çıkan özellikleri düzenli paylaşımlarla tanıtılacaktır.
3. **YouTube Demo Videosu:** Uygulamanın temel özelliklerini gösteren kısa tanıtım videosu hazırlanacaktır.
4. **Product Hunt Lansmanı:** Ürün uluslararası kullanıcı kitlesine tanıtılacaktır.
5. **Reddit Toplulukları:** r/kanban, r/productivity ve r/webdev gibi topluluklarda paylaşım yapılacaktır.
6. **Webrazzi ve Yerel Teknoloji Toplulukları:** Türkiye’deki girişimcilik ve teknoloji platformlarında tanıtım yapılacaktır.

İçerik Pazarlama Stratejisi

Kullanıcı kazanımını desteklemek amacıyla aşağıdaki içerikler hazırlanacaktır:

- “Vanilla JavaScript ile Kanban Uygulaması Geliştirme” blog yazısı
- Uygulama kullanım rehberi
- Özellik karşılaştırma tabloları
- Kısa sosyal medya videoları

Viral Büyüme Mekanizmaları

Organik büyümeyi desteklemek amacıyla aşağıdaki mekanizmalar planlanmaktadır:

- PDF raporlarında ürün bağlantısının yer alması
- JSON export dosyalarında uygulama bilgisi bulunması
- “Bu panoyu paylaş” özelliği
- Kullanıcı referans bağlantıları

Başarı Metrikleri

İlk 3 ay için hedeflenen temel metrikler:

- 100+ kayıtlı kullanıcı
- 25+ aktif haftalık kullanıcı
- %5 üzeri Pro plan dönüşüm oranı
- %3’ün altında aylık churn oranı

Genel Değerlendirme

Bu GTM stratejisi, düşük maliyetli organik büyüme yaklaşımına dayanmaktadır. Açık kaynak paylaşımı, teknik içerik üretimi ve topluluk odaklı tanıtım sayesinde ürünün ilk kullanıcı kitlesine ulaşması ve geri bildirimler doğrultusunda geliştirilmesi hedeflenmektedir. Freemium gelir modeli ile birlikte bu strateji, ürünün sürdürülebilir şekilde büyümesine temel oluşturmaktadır.

10. UYGULAMA VE GELİŞTİRME

10.1. Kurulum ve Çalıştırma

Bu bölümde, Vanilla JS Kanban uygulamasının yerel geliştirme ortamında çalıştırılması için gerekli kurulum adımları açıklanmaktadır. Uygulama tamamen istemci taraflı geliştirildiği için karmaşık bir kurulum süreci gerektirmemektedir. Kaynak kodun indirilmesi ve `index.html` dosyasının bir web sunucusu üzerinden açılması uygulamayı çalıştırmak için yeterlidir.

Sistem Gereksinimleri

Uygulamanın çalıştırılması için aşağıdaki bileşenler yeterlidir:

- Modern bir web tarayıcısı (Chrome, Firefox, Edge veya Safari)
- İsteğe bağlı olarak [Visual Studio Code](#)
- İsteğe bağlı olarak [Git](#)
- İsteğe bağlı olarak basit bir yerel web sunucusu

Kaynak Kodun İndirilmesi

GitHub deposunu klonlamak için aşağıdaki komut kullanılabilir:

```
git clone https://github.com/zekeriyaalkann/kanban-board.git
cd kanban-board
```

Alternatif olarak proje ZIP dosyası olarak indirilebilir ve açılabilir.

Uygulamanın Çalıştırılması

Yöntem 1: Live Server ile

Visual Studio Code içerisinde [Live Server](#) eklentisi kullanılarak `index.html` dosyası doğrudan çalıştırılabilir.

Yöntem 2: Python HTTP Server ile

```
python -m http.server 8000
```

Ardından tarayıcıda aşağıdaki adres açılır:

```
http://localhost:8000
```

İlk Çalıştırma

Uygulama ilk açıldığında varsayılan pano verileri oluşturulur ve `localStorage` üzerinde saklanır. Kullanıcı daha sonra:

- yeni kolonlar ekleyebilir,
- görev kartları oluşturabilir,
- JSON/PDF export işlemleri yapabilir,
- dark/light tema arasında geçiş yapabilir.

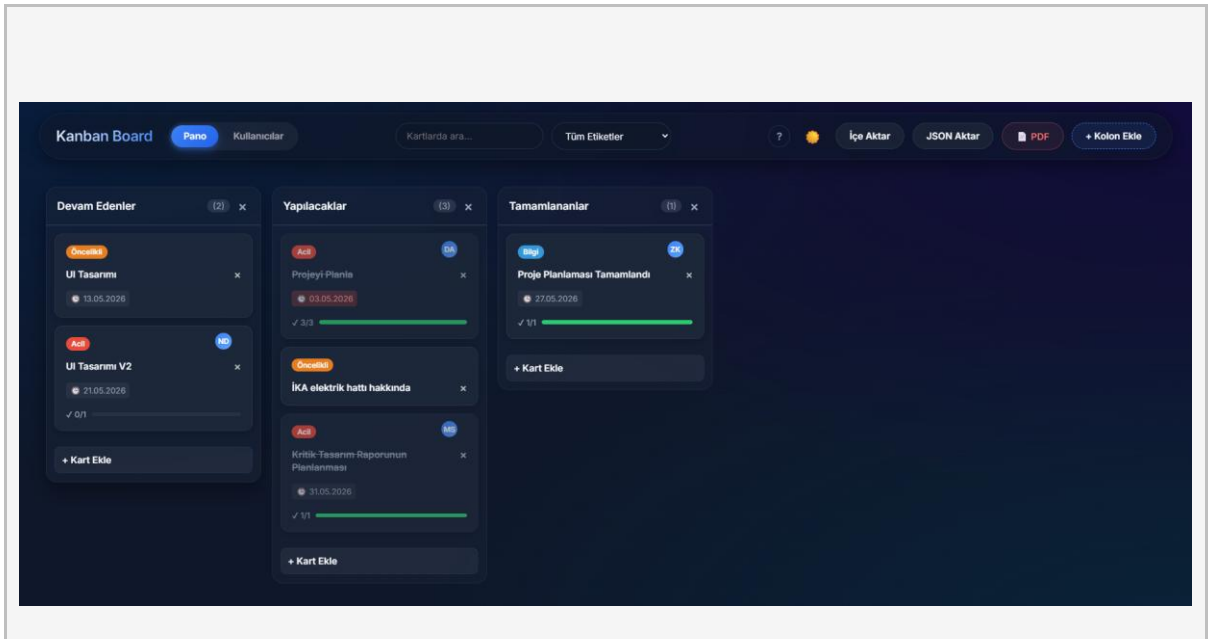
Klasör Yapısı

```
kanban-board/
├── index.html
├── css/
├── js/
├── assets/
├── docs/
└── README.md
```

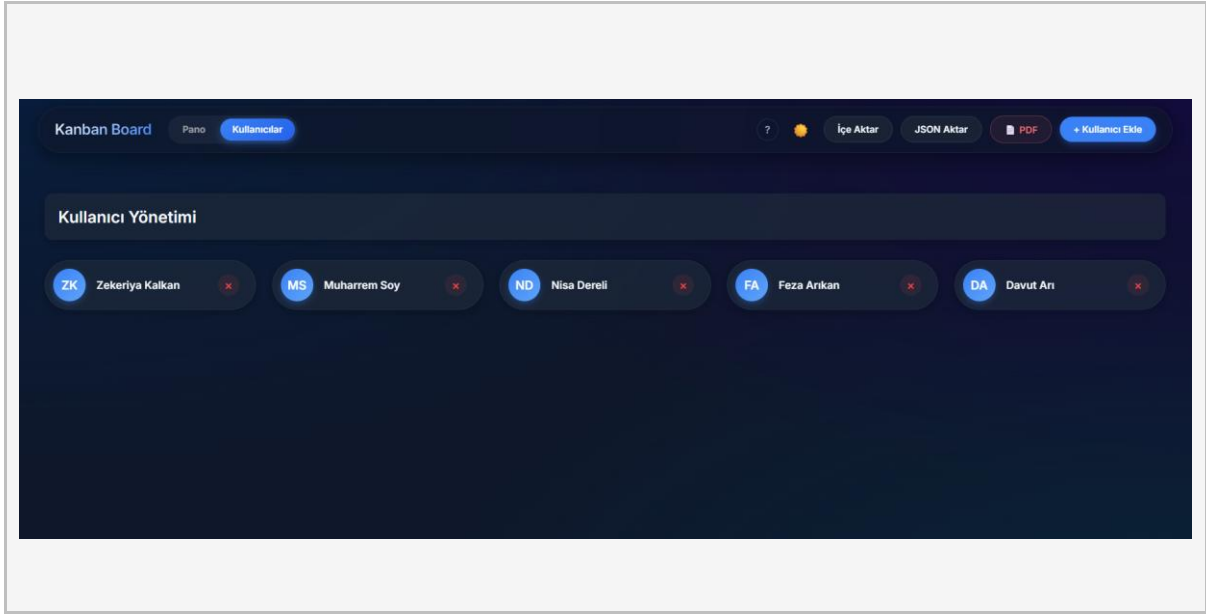
Genel Değerlendirme

Uygulama, herhangi bir backend bağımlılığı gerektirmediğinden kurulumu son derece basittir. Bu yaklaşım, hem geliştirme sürecini hızlandırmakta hem de kullanıcıların ürünü birkaç dakika içinde çalıştırabilmesini sağlamaktadır.

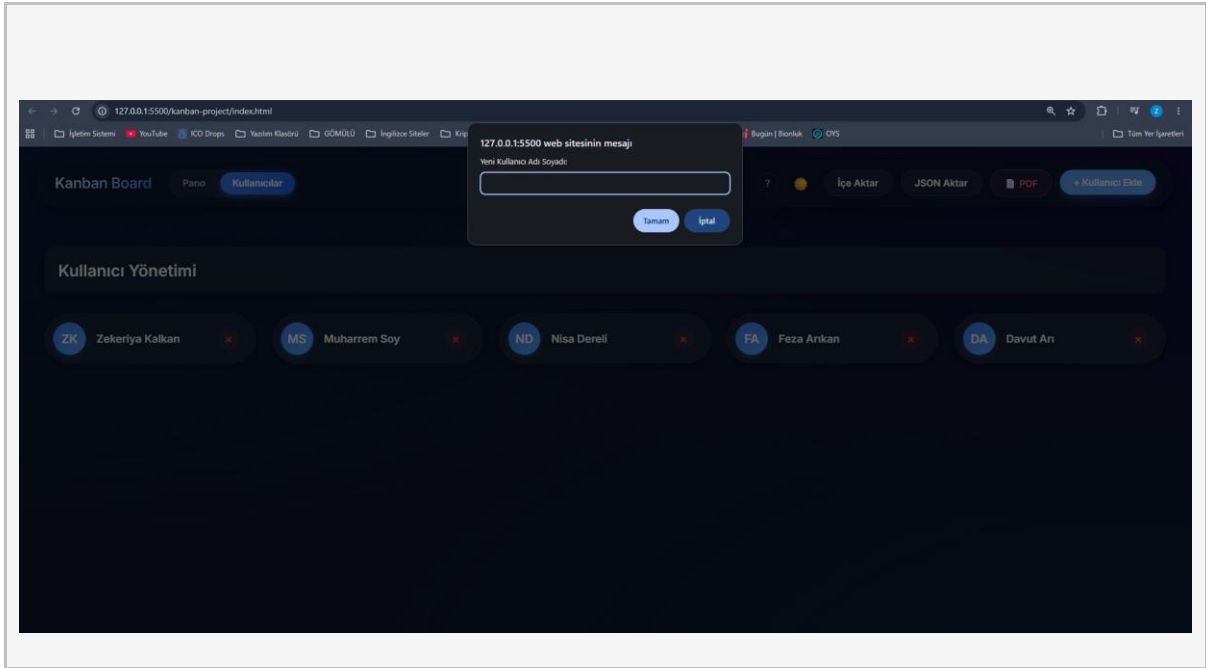
10.2. Ekran Görüntüleri



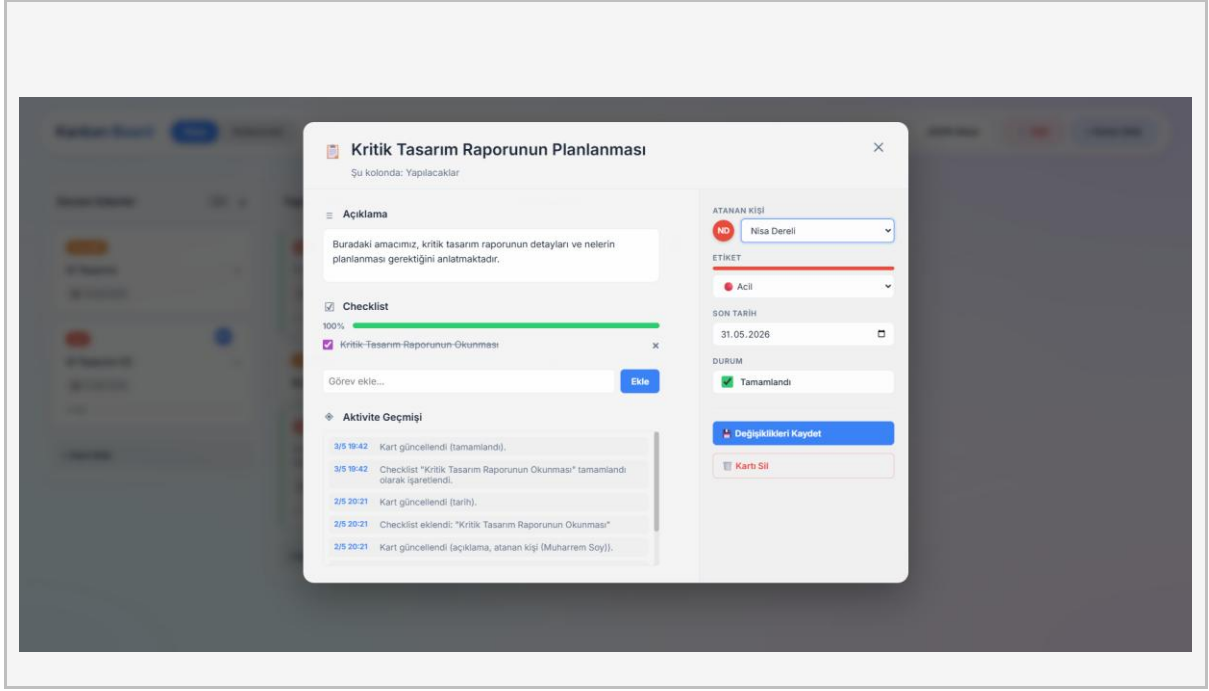
Şekil 11: Landing / Ana Sayfa



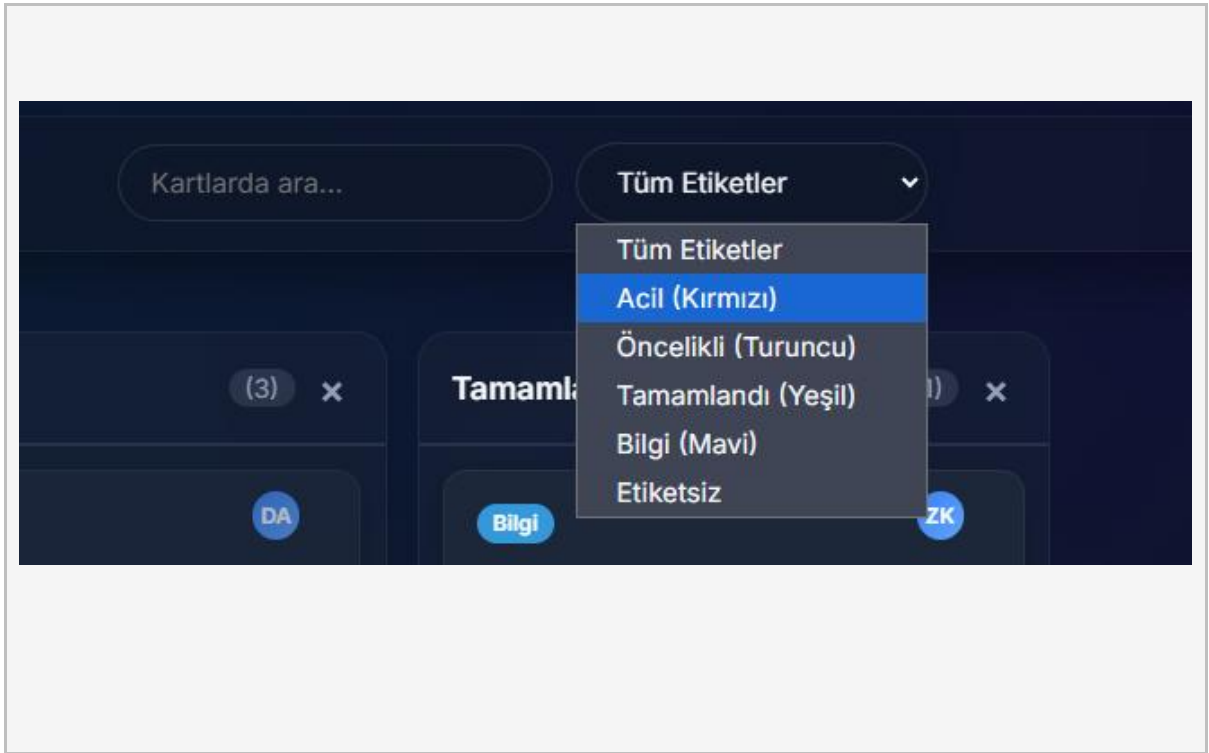
Şekil 12: Kayıt / Giriş Ekranı



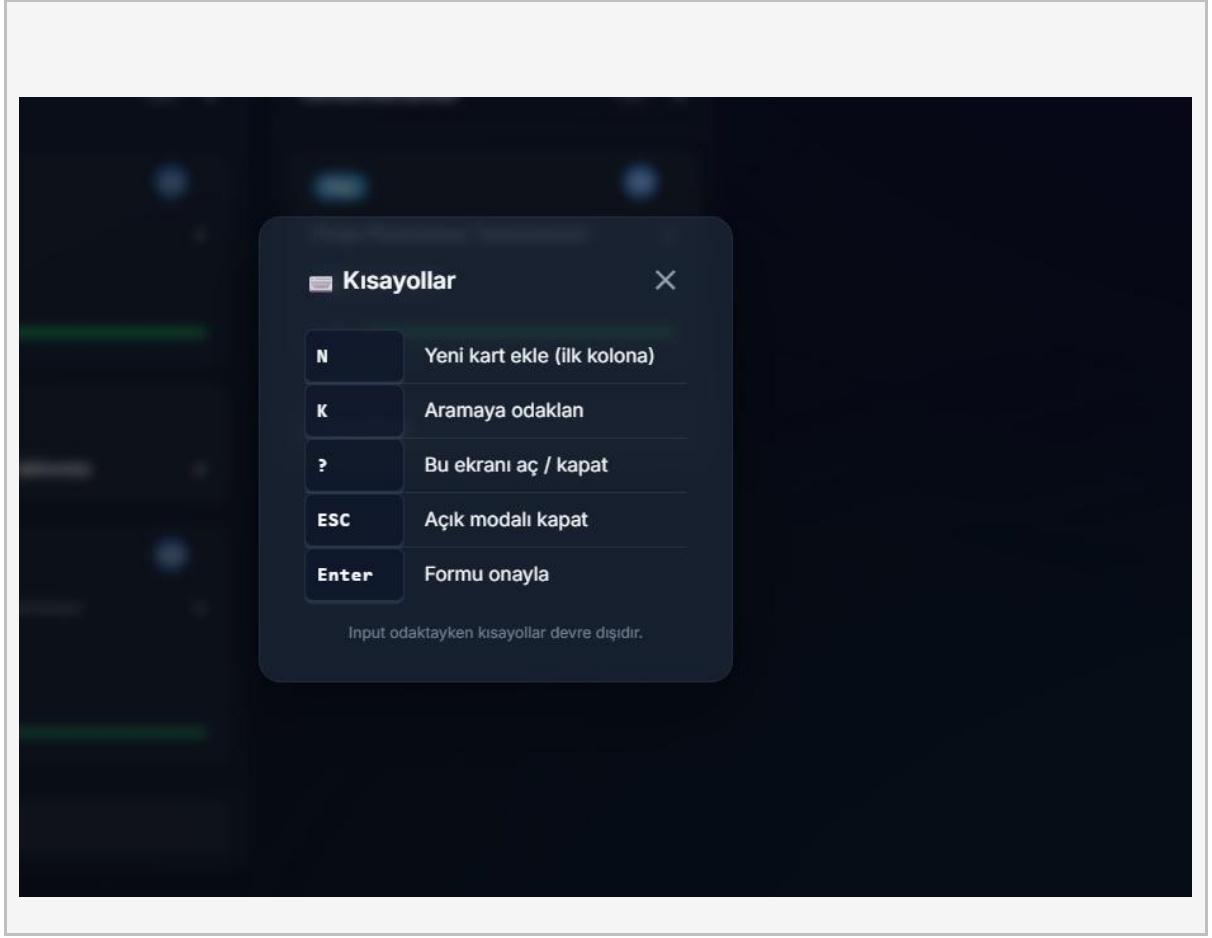
Şekil 13: Dashboard — Boş State



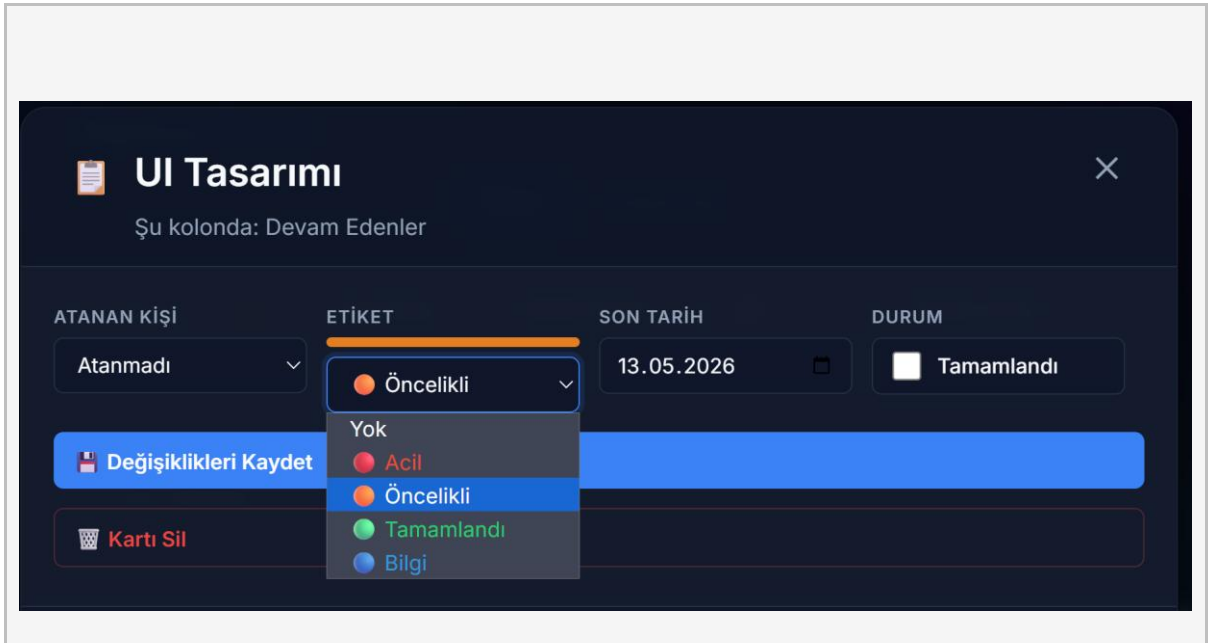
Şekil 14: Light Mode Görünüm



Şekil 15: Ana Filtreleme Mekanizması



Şekil 16: Kısayol Gösterimleri



Şekil 17: Mobile Görünüm

Projeyi Planla

Şu kolonda: Yapılacaklar

Açıklama

İKA için elektrik hattı, görüntü işleme ve mekanik tasarımın bitmesini istiyoruz.

Checklist

100%

Elektrik hattı ön çiziminin yapılması

Elektrik hattı ön çiziminin test edilmesi

Elektrik hattı çiziminin nihai olarak tamamlanması

Görev ekle...

Ekle

Aktivite Geçmişi

4/5 15:40 Kart güncellendi (atanan kişi (Davut Arı)).

3/5 19:42 Kart güncellendi (tamamlandı).

3/5 19:42 Checklist "Elektrik hattı çiziminin nihai olarak tamamlanması" tamamlandı olarak işaretlendi.

3/5 19:42 Checklist "Elektrik hattı ön çiziminin test edilmesi" tamamlandı olarak işaretlendi.

3/5 19:01 Kart güncellendi (tamamlanmadı).

ATANAN KİŞİ

DA Davut Arı

ETİKET

Acil

SON TARİH

03.05.2026

DURUM

Tamamlandı

Değişiklikleri Kaydet

Kartı Sil

Şekil 18: Detaylı Kart Gösterimi

10.3. Kullanılan AI Araçları ve Katkı Oranı

Tablo 37: AI Kullanım Dağılımı

Araç	Kullanım Oranı	Ne için?
Claude Code (Anthropic)	%10	Kod inceleme, metin iyileştirme ve ek teknik değerlendirmeler
ChatGPT (OpenAI)	%50	Dokümantasyon hazırlama, rapor yazımı, teknik açıklamalar, mimari kararların oluşturulması
Gemini (Google)	%15	Alternatif çözüm önerileri, araştırma ve teknik doğrulama
Antigravity	%25	Kod analizi, refactoring önerileri, hata

		ayıklama ve geliştirme desteği
AI çıktısının gözden geçirilmesi	%100	Tüm AI çıktıları manuel olarak incelenmiş, doğrulanmış ve proje gereksinimlerine göre düzenlenmiştir

11. SONUÇ VE DEĞERLENDİRME

11.1. Proje Hakkında Genel Değerlendirme

Bu proje kapsamında, tamamen istemci taraflı çalışan ve herhangi bir backend bağımlılığı gerektirmeyen bir Kanban görev yönetim uygulaması geliştirilmiştir. Uygulama; görev kartlarının oluşturulması, düzenlenmesi, silinmesi, sürükle-bırak yöntemiyle taşınması, kullanıcı ataması, checklist yönetimi, JSON import/export, PDF raporlama, dark/light tema desteği ve klavye kısayolları gibi birçok modern özelliği içermektedir.

Proje başlangıcında belirlenen temel hedef, saf HTML5, CSS3 ve Vanilla JavaScript kullanarak profesyonel seviyede, modüler ve genişletilebilir bir görev yönetim sistemi geliştirmektir. Bu hedef doğrultusunda uygulamanın kullanıcı arayüzü, veri modeli, sistem mimarisi, performans hedefleri, güvenlik değerlendirmeleri ve ürün stratejisi detaylı şekilde tasarlanmış ve dokümanite edilmiştir.

Planlanan fonksiyonel gereksinimlerin büyük bölümü başarıyla gerçekleştirilmiştir. Özellikle aşağıdaki hedefler tam olarak karşılanmıştır:

- Kanban panosunun dinamik olarak oluşturulması,
- Kolon ve kart yönetimi,
- HTML5 Drag & Drop desteği,
- localStorage ile veri kalıcılığı,
- JSON import/export mekanizması,
- jsPDF ile PDF raporlama,
- Responsive tasarım,
- Dark/light tema desteği,
- Erişilebilirlik prensiplerinin uygulanması.

Performans açısından uygulama hızlı açılmakta ve kullanıcı etkileşimlerine akıcı şekilde yanıt vermektedir. Düşük bağımlılık yaklaşımı sayesinde kaynak kullanımı minimum düzeyde tutulmuştur. Kullanılabilirlik bakımından sistem masaüstü ve mobil cihazlarda sorunsuz şekilde çalışmaktadır.

Başlangıçta planlanan ancak mevcut sürümde tam olarak uygulanmayan veya geleceğe bırakılan özellikler ise şunlardır:

- Çok kullanıcılı çalışma yapısı,
- Gerçek zamanlı eşzamanlama,
- Sunucu tabanlı veritabanı,
- Kimlik doğrulama ve yetkilendirme sistemi,
- Bulut senkronizasyonu.

Genel olarak değerlendirildiğinde proje, belirlenen eğitim ve teknik hedefleri başarıyla karşılamıştır. Bunun yanında, sadece çalışan bir yazılım ürünü ortaya konmamış; ürün analizi, sistem mimarisi, veri modelleme, güvenlik, performans ve iş modeli boyutlarını kapsayan kapsamlı bir mühendislik dokümantasyonu hazırlanmıştır.

Bu çalışma sonucunda modern web teknolojileri, yazılım mimarisi, teknik raporlama ve ürün geliştirme süreçleri hakkında önemli deneyimler kazanılmıştır. Elde edilen çıktı, hem akademik değerlendirme hem de profesyonel portföy açısından güçlü bir referans niteliği taşımaktadır.

11.2. Karşılaşılan Zorluklar ve Çözümleri

Tablo 38: Geliştirme Sürecindeki En Büyük 3 Zorluk

Zorluk	Çözüm	Öğrenilen
Sürükle-bırak işlemlerinde kartların farklı kolonlara taşınırken veri yapısının bozulması	HTML5 Drag & Drop olayları yeniden düzenlendi, bırakma sonrasında App.data güncellenerek arayüz yeniden render edildi	UI işlemleri ile veri modelinin her zaman senkron tutulması gerektiği öğrenildi
localStorage üzerinde saklanan verilerin farklı sürümler arasında uyumsuz hale gelmesi	Varsayılan veri şeması tanımlandı, import sırasında veri doğrulama ve hata kontrolü eklendi	Kalıcı veri yapılarında şema tutarlılığı ve doğrulama mekanizmalarının önemi görüldü
Büyük ve kapsamlı bir raporun hazırlanması sırasında içeriklerin organize edilmesi ve teknik kararların sistematik biçimde dokümante edilmesi	PRD, ADR, C4 diyagramları ve tablo tabanlı şablon kullanılarak rapor adım adım oluşturuldu	Yazılım geliştirme sürecinde dokümantasyonun en az kod kadar önemli olduğu ve proje kalitesini doğrudan artırdığı öğrenildi

11.3. Gelecek Çalışmalar (Future Work)

V2 hedefleri: Kullanıcı hesapları + cloud sync (Supabase), takım kolaborasyonu

V3 hedefleri: Mobile app (Capacitor), AI öncelik önerisi

11.4. Kazanılan Yetkinlikler

Bu proje sürecinde yalnızca işlevsel bir web uygulaması geliştirilmemiş, aynı zamanda yazılım mühendisliğinin ürün yönetimi, mimari tasarım, güvenlik, performans ve dokümantasyon boyutlarında önemli deneyimler kazanılmıştır. Proje boyunca elde edilen başlıca teknik ve süreçsel kazanımlar aşağıda özetlenmiştir.

Teknik Kazanımlar

1. **Vanilla JavaScript ile Modüler Uygulama Geliştirme**
Framework kullanmadan modern JavaScript özellikleri ile sürdürülebilir ve modüler bir uygulama mimarisi oluşturma deneyimi kazanılmıştır.
2. **HTML5 Drag & Drop API Kullanımı**
Kartların sürükle-bırak yöntemiyle taşınmasını sağlayan etkileşimli kullanıcı arayüzlerinin geliştirilmesi öğrenilmiştir.
3. **localStorage Tabanlı Veri Yönetimi**
Tarayıcı üzerinde kalıcı veri saklama, JSON serileştirme ve veri geri yükleme mekanizmaları uygulanmıştır.
4. **JSON Import / Export ve PDF Raporlama**
Kullanıcı verilerinin yedeklenmesi, içe aktarılması ve PDF formatında raporlanması süreçleri gerçekleştirilmiştir.
5. **Responsive ve Erişilebilir Arayüz Tasarımı**
CSS Grid, Flexbox, media query, ARIA ve WCAG prensipleri kullanılarak farklı cihazlara uyumlu ve erişilebilir bir arayüz geliştirilmiştir.
6. **Performans ve Güvenlik Değerlendirmesi**
Web Vitals, OWASP Top 10 ve temel güvenlik kontrolleri kullanılarak uygulamanın kalite özellikleri analiz edilmiştir.

Süreçsel Kazanımlar

1. **PRD (Product Requirements Document) Hazırlama**
Problem tanımı, kullanıcı ihtiyaçları, başarı metrikleri ve ürün kapsamının sistematik şekilde dokümanite edilmesi öğrenilmiştir.
2. **Persona ve JTBD Analizi**
Kullanıcı odaklı düşünme yaklaşımıyla hedef kitlenin ihtiyaçlarının daha net tanımlanması sağlanmıştır.
3. **Architecture Decision Records (ADR)**
Alınan teknik kararların gerekçeleriyle birlikte kayıt altına alınmasının önemi kavranmıştır.
4. **C4 Modeli ile Sistem Mimarisi Dokümantasyonu**
Sistem mimarisinin context, container ve sequence seviyelerinde görselleştirilmesi deneyimlenmiştir.
5. **Business Model Canvas ve Unit Economics**
Bir yazılım ürününün teknik boyutunun yanı sıra iş modeli, maliyet yapısı ve gelir potansiyelinin değerlendirilmesi öğrenilmiştir.
6. **Kapsamlı Teknik Raporlama**
Kodun ötesinde, ürün geliştirme sürecinin profesyonel düzeyde dokümanite edilmesinin proje kalitesine olan katkısı görülmüştür.

Genel Değerlendirme

Bu proje, modern web teknolojileriyle uygulama geliştirme becerilerini güçlendirmenin yanı sıra; sistematik düşünme, teknik kararları gerekçelendirme, kullanıcı odaklı tasarım ve ürün stratejisi oluşturma konularında önemli kazanımlar sağlamıştır. Elde edilen

yetkinlikler, gelecekte gerçekleştirilecek akademik ve profesyonel projeler için güçlü bir temel oluşturmuştur.

KAYNAKÇA

- [1] Mozilla Developer Network, “HTML: HyperText Markup Language,” Available: <https://developer.mozilla.org/en-US/docs/Web/HTML>
- [2] Mozilla Developer Network, “CSS: Cascading Style Sheets,” Available: <https://developer.mozilla.org/en-US/docs/Web/CSS>
- [3] Mozilla Developer Network, “JavaScript Guide,” Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>
- [4] Mozilla Developer Network, “Web Storage API,” Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
- [5] Mozilla Developer Network, “HTML Drag and Drop API,” Available: https://developer.mozilla.org/en-US/docs/Web/API/HTML_Drag_and_Drop_API
- [6] Parallax, “jsPDF Documentation,” Available: <https://github.com/parallax/jsPDF>
- [7] Google Chrome Developers, “Lighthouse Overview,” Available: <https://developer.chrome.com/docs/lighthouse/overview/>
- [8] OWASP Foundation, “OWASP Top 10: The Ten Most Critical Web Application Security Risks,” Available: <https://owasp.org/www-project-top-ten/>
- [9] World Wide Web Consortium (W3C), “Web Content Accessibility Guidelines (WCAG) 2.1,” Available: <https://www.w3.org/TR/WCAG21/>
- [10] Git, “Git Documentation,” Available: <https://git-scm.com/doc>
- [11] GitHub, “GitHub Docs,” Available: <https://docs.github.com/>
- [12] Microsoft, “Visual Studio Code Documentation,” Available: <https://code.visualstudio.com/docs>
- [13] ECMA International, “ECMAScript® 2024 Language Specification,” Available: <https://tc39.es/ecma262/>
- [14] OpenAI, “ChatGPT,” Available: <https://chatgpt.com/>
- [15] Anthropic, “Claude,” Available: <https://www.anthropic.com/claude>
- [16] Google, “Gemini,” Available: <https://gemini.google.com/>
- [17] Antigravity, “Antigravity AI Coding Assistant,” Available: <https://www.antigravity.dev/>